

JAVA SDE-2 INTERVIEW PREPARATION SERIES

SYSTEM DESIGN - BOOK 04 OF 05 - STUDY STEP SD-04

Low-Level Design and Machine Coding

Modeling, State Machines, Invariants, and Extensible Java 21 for the Machine-Coding Round

BY

Vinay Reddy Kalluri

Design and implement eight classic machine-coding problems under time pressure: scope in writing, separate entities from values and policies, name the concurrency owner, protect every invariant, and absorb the extension round with one new class.

METHOD | PARKING LOT | STATE MACHINES | CACHES | RATE LIMITERS | LEDGERS | SCHEDULERS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [SD 03 - Generative AI System Design](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This book covers the low-level design and machine-coding round, where experienced engineers most often produce working code and still fail. It teaches a fixed method and applies it to eight problems - parking lot, vending machine, elevator, LRU and LFU caches, rate limiter, split expenses, job scheduler, and notification service - with every claim checked by an executable Java 21 model.

Readiness map

Decision	Evidence
START HERE	The prompt says design and implement, not only design; A runnable program or tests are expected within 60 to 120 minutes; The problem has a lifecycle, a policy that is likely to change, or a shared resource; The interviewer adds a requirement to working code near the end.
PREPARE FIRST	SD-01, or equivalent comfort with interfaces, records, enums, and basic design principles; JAVA-05 collections and JAVA-07 concurrency for the thread-safety sections; No framework knowledge is assumed; every solution is plain Java 21.
MOVE ON WHEN	Run a fixed six-step method with a written scope and a back-loaded time budget; Separate entities, value objects, and policies so an extension lands in one new type; Name the concurrency owner for every invariant and prove it with a concurrent test; Implement caches, rate limiters, ledgers, and schedulers with their invariants checked; Answer the concurrency, persistence, scale, and failure follow-ups that close the round.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

System Design Segment Position

SD 03 - Generative AI System Design	YOU ARE HERE SD 04 - Low-Level Design and Machine Coding	SD 05 - High-Level Design Case Book
-------------------------------------	--	-------------------------------------

A note from Vinay

Working code is the entry ticket in this round, not the score. Write the scope down before you write a class, keep the rule that will change behind an interface, and say out loud which object owns each invariant when two requests arrive at once. The interviewer will add a requirement in the last twenty minutes; if it costs you one new class, you designed well.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - The Machine-Coding Round: Method and Rubric	4
Chapter 2 - Parking Lot: Allocation, Pricing, and Concurrent Entry	11
Chapter 3 - State Machines: Vending Machine and Elevator	17
Chapter 4 - Caches and Rate Limiters: Data Structure Plus Policy	23
Chapter 5 - Split Expenses: Money, Invariants, and Settlement	30
Chapter 6 - Job Scheduler and Notification Service: Lifecycle and Retries	36
Chapter 7 - Extension Rounds, Follow-Ups, and Question Bank	43
Chapter 8 - Low-Level Design and Machine-Coding Drills	50
Chapter 9 - Low-Level Design and Machine-Coding Drills: Worked Solutions	52
Chapter 10 - Executable Low-Level Design Model	58
Part II - Practice and Handoff	105

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - The Machine-Coding Round: Method and Rubric

Learning objectives

By the end of this chapter, you should be able to:

- describe what a machine-coding or low-level design round is actually scoring, and why "it works" is necessary but nowhere near sufficient;
- run a fixed six-step method inside a 60 to 120 minute budget without losing the last 20 minutes to debugging;
- separate entities, value objects, policies, and state so that a new requirement lands in one place;
- decide where concurrency is owned before writing the first class; and
- recognise the three most common ways a strong engineer still fails this round.

WHY IT WORKS | Why this matters at SDE-2

The low-level design round goes by several names - machine coding, object-oriented design, LLD - and it is the round where experienced engineers are most often surprised by the result. They produce code that runs and still get a "no hire" or a down-level to SDE-1.

The reason is that the round is not testing whether you can make a parking lot work. Almost everyone can. It is testing whether the code you produce under time pressure is the code a senior teammate would approve: responsibilities in the right places, rules that can change without a rewrite, invariants that cannot be violated from outside, and a clear answer to "what happens when two requests arrive at once?"

At SDE-1, working code with reasonable names is a pass. At SDE-2, the interviewer expects you to *drive*: to scope the problem yourself, to name the extension points before being asked, and to defend each boundary. A candidate who waits to be told what to build next is showing SDE-1 behaviour regardless of how clean the code is.

WHY IT WORKS | First-principles model

A machine-coding problem is a small domain with three kinds of content, and good designs keep them separate:

Things with identity that change over time (entities). A parking spot, a ticket, a job, an elevator car. They have an id, a lifecycle, and state transitions. Two tickets with identical fields are still two tickets.

Values that mean what they contain (value objects). A money amount, a time range, a coin, a floor number. They are immutable and compared by content. In Java 21 they are almost always **records**.

Rules that are expected to change (policies). How a spot is chosen, how a stay is priced, which cache entry is evicted, which channel a notification uses. Every problem in this book has at least one rule the interviewer will change in the last 20 minutes. If that rule is an **if** statement buried inside the entity, the change is a rewrite; if it is an interface with one implementation, the change is a new class.

On top of these sits one question that decides whether the design survives contact with reality: **who owns concurrency?** Pick the single object whose lock (or atomic operation) makes each invariant true, and say it out loud. "The lot's **park** method holds the lock across choosing and claiming a spot" is an answer. "We'll make things thread-safe" is not.

Specification boundary: there is no standard for this round. Formats vary by company: some want a runnable program with a **main** demo, some want tests, some accept pseudo-code for the non-critical paths, and some let you use an IDE with autocomplete. Ask in the first minute. Everything in this book assumes the strictest common format - runnable Java 21, no frameworks, one to two hours - because a design that meets that bar also meets the looser ones.

Core terminology

- **Machine coding:** a timed round where you design *and implement* a small system that must run, usually with a demo driver or tests.
- **Entity:** an object with identity and a lifecycle; equality by id.
- **Value object:** an immutable object compared by content; a **record** in modern Java.
- **Policy (strategy):** an interface isolating a rule that is expected to vary.
- **Invariant:** a condition that must hold after every public operation, such as "no spot is assigned to two vehicles".
- **State machine:** an explicit set of states and permitted transitions; illegal transitions are rejected, not silently ignored.
- **Extension round:** the part of the interview, usually the last 15 to 25 minutes, where a new requirement is added to your working code.
- **Critical path:** the one flow that must work end to end, such as park then exit then pay.

Detailed mechanics

What the rubric actually scores

Interviewers rarely share their rubric, but they converge on the same five dimensions. Ordered by how often each one decides the outcome:

Dimension	What a pass looks like	What a fail looks like
Working critical path	The main flow runs end to end in a demo or test	Code that "would work" but was never executed
Separation of concerns	Entities, values, and policies in distinct types	One <code>ParkingLotService</code> with 600 lines and a <code>switch</code>
Extensibility	The new requirement lands in one new class	The new requirement touches five files
Correctness under concurrency	The lock owner is named and the check-then-act is atomic	<code>if (spot.isFree()) spot.occupy()</code> from two threads
Communication	Scope, trade-offs, and shortcuts are stated aloud	Silent typing for 40 minutes

Two observations follow. First, "working" is the *entry ticket*, not the score. Second, three of the five dimensions are visible only if you narrate them. An interviewer cannot credit a trade-off you made silently.

The six-step method

Use the same method every time. The point of a fixed method is that under pressure you stop deciding *what to do next* and spend all of your attention on the problem.

TEXT		
step	budget (of 90 min)	artifact
1. clarify and scope	5-8 min	written list of in-scope use cases and non-goals
2. model the domain	8-10 min	entities, values, policies, states - names only
3. define the interfaces	5 min	method signatures for the critical path
4. implement the critical path	30-35 min	running code, demo in main or a test
5. harden	10-15 min	invariants, edge cases, concurrency owner
6. extend and discuss	15-20 min	the new requirement, then trade-offs

The budget is deliberately back-loaded. Candidates who spend 25 minutes on class diagrams run out of time to make anything run; candidates who start typing in minute two build the wrong thing and restructure in minute 50. The first 20 minutes produce no code, and that is correct.

Step 1: clarify and scope. Write down, in the shared editor, three to six use cases you will implement and two or three you explicitly will not. For a parking lot: *park a vehicle, exit and pay, show free spots by type* - in scope; *reservations, multiple entrances with separate displays, payment gateway integration* - out of scope, with a sentence on where each would plug in. This list is your contract with the interviewer and your defence when the clock runs out.

Step 2: model the domain. Nouns become candidate types; verbs become candidate methods. Then prune: a noun that has no behaviour and no identity is a value or a field, not a class. Mark which rules are likely to change - those become interfaces.

Step 3: define the interfaces. Write the public signatures for the critical path before any bodies. `Optional<Ticket> park(String plate, VehicleType type)` and `long exit(long ticketId)` tell the interviewer, and you, what "done" means.

Step 4: implement the critical path. One flow, end to end, runnable. Resist implementing the second use case until the first one prints the right answer. A stubbed pricing policy that returns a constant is fine at this stage; a lot that cannot park anything is not.

Step 5: harden. Add the invariant checks, the illegal-transition rejections, and the lock. This is where most of the SDE-2 signal is earned, because it is where you show what can go wrong.

Step 6: extend and discuss. The interviewer adds a requirement. If your policies are interfaces, you write a class and change one constructor argument. Then discuss what you would change at 100 times the scale.

Modeling toolkit in Java 21

Java 21 gives you exactly the tools this round rewards, and using them is itself a signal:

JAVA

```
enum VehicleType {
    MOTORCYCLE(SpotSize.SMALL), CAR(SpotSize.COMPACT), TRUCK(SpotSize.LARGE);

    final SpotSize minimumSpot;

    VehicleType(SpotSize minimumSpot) {
        this.minimumSpot = minimumSpot;
    }

    boolean fits(SpotSize size) {
        return size.ordinal() >= minimumSpot.ordinal();
    }
}

record Spot(String id, int level, SpotSize size) { }

interface AllocationPolicy {
    Optional<Spot> choose(List<Spot> freeSpots, VehicleType vehicle);
}
```

- **enum with behaviour** for closed sets whose members carry rules (`VehicleType.fits`). Adding a vehicle type is then one line plus its rule, and the compiler finds every `switch` that needs updating.
- **record** for values. Equality, hashing, and immutability come free, and a reviewer immediately knows the type has no lifecycle.
- **sealed interface with record implementations** for closed sets of *shapes*, such as commands or events, where pattern-matching `switch` gives exhaustiveness checking.
- **Plain interfaces** for policies. One method is usually enough.

- **Optional** as the return type of operations that can legitimately find nothing (**park** when the lot is full). Exceptions are for broken contracts, such as exiting with an unknown ticket.

Where concurrency is owned

Every problem in this book has a check-then-act sequence that is wrong under concurrency: check that a spot is free, then claim it; check the balance, then deduct; check the cache, then load. The fix is always the same shape - make the check and the act one atomic step - but *which object provides the atomicity* is a design decision you must make and state.

Three defensible options, in increasing order of complexity:

- 1 **One coarse lock on the aggregate.** **synchronized** on **ParkingLot.park** and **exit**. Correct, easy to reason about, and fast enough for any interview-scale workload. Start here.
- 2 **Per-partition locks.** A lock per floor or per spot size, when the coarse lock is shown to be contended. Requires that every invariant lives inside one partition.
- 3 **Lock-free claim.** A **ConcurrentLinkedQueue** of free spots, or **compareAndSet** on a spot's state. Fastest, but every multi-step invariant now needs a separate argument.

The companion model runs 8 threads making 8,000 park attempts against 500 spots with option 1, and checks the result: exactly 500 successes, 500 distinct spots, 0 free. A version that checked **isFree()** and then claimed outside the lock can assign one spot twice; the point of the test is that the correct version *cannot*.

Anti-patterns that cost the round

Pattern stuffing. Naming five design patterns in the first ten minutes, then building a **VehicleFactoryFactory**. A pattern is justified by a rule that varies, not by the problem having a noun. The parking lot needs one strategy for allocation and one for pricing; it does not need an abstract factory, a visitor, and an observer.

The god service. One **XService** class that owns all state and all rules. It usually starts as a reasonable shortcut and becomes impossible to extend in step 6.

Anemic entities with a smart service. Entities that are only getters and setters, and a service that reaches into them to enforce every rule. The invariant "a ticket can be closed once" belongs to whatever owns tickets, not to every caller.

Premature generality. An interface for every class "in case". An interface with one implementation is justified only when you can name the second implementation the interviewer is likely to ask for.

Silent shortcuts. Using **double** for money because it is faster to type, without saying so. The shortcut itself is sometimes fine; failing to name it reads as not knowing.

WATCH OUT | Failure modes and common mistakes

- **Running out of time with nothing running.** The most common failure. Enforce the step budget and get the critical path executing by minute 45 even if it is ugly.
- **Asking no clarifying questions.** The prompt is deliberately underspecified. Building the wrong scope is scored as a judgement failure, not bad luck.
- **Over-clarifying.** Ten minutes of questions with no written scope. Clarify, write it down, move on.
- **Untested demo.** Printing "parked" without checking which spot was taken. Assert the result, even if only with a `check(condition, message)` helper.
- **Treating the extension as a surprise.** It is not. For every problem in this book, the likely extensions are listed; design the seam before being asked.
- **Defending a bad decision.** When the interviewer points at a flaw, the strongest response is to agree, state the fix and its cost, and decide whether to apply it now or note it.

TRY IT | Interview questions and model answers

Q: "Design a parking lot." What is your first move? A: Scope it in writing. I would propose: park, exit with pricing, and free-spot counts by size are in scope; reservations, payment integration, and multiple entry displays are out, and I will say where each would plug in. I would also ask whether they want a runnable program or tests, and how many entrances there are, because concurrent entry changes the design.

Q: Why an interface for pricing when there is only one pricing rule? A: Because pricing is the rule most likely to change - weekend rates, event pricing, per-vehicle-type rates - and I can name the second implementation now. Allocation is the same. I would not add an interface for `Ticket`, because there is no second kind of ticket I can name.

Q: Where does thread safety live in your design? A: In the aggregate that owns the invariant. The lot's `park` method holds its lock across choosing and removing the spot from the free list, so two entrances can never receive the same spot. I start with one coarse lock because it is correct and simple, and would partition it by floor only if measurements showed contention.

Q: You used `double` for price. Is that a problem? A: Yes, and I should have said so. Binary floating point cannot represent most decimal cents, so sums drift - 100 divided by 3 rounded to cents is 33.33, and three of those make 99.99. I would store money as `long` cents, or `BigDecimal` with an explicit scale and rounding mode if fractional cents are meaningful.

Q: How do you know your design is extensible? A: I test it against the extensions I expect. For each likely requirement I check how many types it touches. If a new pricing rule needs one new class and a constructor change, the seam is right. If it needs edits in the lot, the ticket, and the display, it is not.

TRY IT | Exercises

- 1 Take any problem from chapters 2 to 6 and write only steps 1 to 3 - scope, domain model, and signatures - in 15 minutes. Compare with the chapter.
- 2 List three rules in a vending machine that are likely to change. For each, write the interface you would introduce and name its second implementation.
- 3 Write a check-then-act bug for a seat-booking system, then fix it in two different ways, and state which object owns concurrency in each fix.

RECAP | Chapter summary

The machine-coding round scores working code as the entry requirement and then grades separation of concerns, extensibility, concurrency, and communication. A fixed six-step method with a back-loaded time budget prevents the two classic failures: nothing running, and the wrong thing running. Model entities, values, and policies as separate kinds of type; name the concurrency owner before writing code; and treat the extension round as predictable rather than as a surprise.

TRY IT | Revision checklist

- ☐ I can list the five scoring dimensions and which three depend on narration.
- ☐ I can run the six-step method with a written scope inside eight minutes.
- ☐ I can classify any noun in a prompt as entity, value, policy, or field.
- ☐ I name the concurrency owner for every invariant before implementing.
- ☐ I state shortcuts aloud, including the ones I would not ship.

SDE-2 CORE CHAPTER

Chapter 2 - Parking Lot: Allocation, Pricing, and Concurrent Entry

Learning objectives

By the end of this chapter, you should be able to:

- model a parking lot with the compatibility rule on the vehicle, not scattered through `if` statements;
- explain why the choice of allocation policy is a correctness decision, not only an optimisation;
- implement pricing as a policy with a grace period, started-hour rounding, and a daily cap, and test its boundaries;
- make concurrent entry correct by construction and prove it with a test; and
- handle the standard extensions: multiple floors, reservations, electric-vehicle spots, and dynamic pricing.

WHY IT WORKS | Why this matters at SDE-2

The parking lot is the most-asked LLD problem, which is exactly why it discriminates well. Every candidate has seen a solution. The interviewer is therefore not asking whether you can produce classes named `Spot` and `Ticket`; they are watching for whether you see the two places where the problem is genuinely hard - allocation under constraints and concurrent entry - and whether your pricing survives its boundary cases.

WHY IT WORKS | First-principles model

A parking lot is a **constrained resource allocator with a billing side effect**.

- The **resource** is a set of spots, each with a size.
- The **constraint** is compatibility: a vehicle may use a spot of at least its minimum size.
- The **allocator** chooses among compatible free spots - and different choices lead to different future rejections.
- The **billing** is a function of time, computed when the vehicle leaves.
- The **invariant** is that no spot is ever held by two vehicles and no ticket is closed twice.

Everything else - displays, gates, payment methods - is presentation around that core.

Core terminology

- **Compatibility rule:** which vehicle types may use which spot sizes.
- **Allocation policy:** the rule that chooses one spot among the compatible free spots.
- **First fit:** take the first compatible spot in some fixed order.
- **Best fit:** take the smallest compatible spot, preserving large spots for large vehicles.
- **Grace period:** a short initial duration that is free.
- **Started hour:** billing that rounds any partial hour up.
- **Daily cap:** the maximum charged for one 24-hour period.

Detailed mechanics

The domain model

TEXT

VehicleType (enum)	MOTORCYCLE, CAR, TRUCK - each knows its minimum SpotSize
SpotSize (enum)	SMALL < COMPACT < LARGE
Spot (record)	id, level, size
Ticket (record)	ticketId, plate, spot, entryMinute
AllocationPolicy	Optional<Spot> choose(List<Spot> free, VehicleType v)
PricingPolicy	long priceCents(long minutesParked)
ParkingLot	owns free spots, active tickets, both policies, and the lock

Note what is *not* a class: there is no `Vehicle` hierarchy with `Car` extends `Vehicle`. The only behaviour that varies by vehicle is compatibility, and an `enum` with a field expresses that in three lines. Introduce a class hierarchy only when vehicle types differ in *behaviour* - which in this problem they do not.

The compatibility rule lives on `VehicleType`:

JAVA

```
boolean fits(SpotSize size) {
    return size.ordinal() >= minimumSpot.ordinal();
}
```

Relying on `ordinal()` encodes "sizes are totally ordered" in the declaration order of `SpotSize`. That is acceptable and worth one sentence aloud. If the interviewer adds a spot type that is not simply larger - an electric-vehicle spot, a disabled-access spot - the rule stops being an order, and `fits` becomes a lookup table from vehicle to the set of acceptable spot types.

Allocation is a correctness decision

It is tempting to treat spot selection as an optimisation to discuss later. It is not: the policy changes *which future vehicles are rejected*.

Take a lot with one large spot **L - 1** declared first and one compact spot **C - 1**. A car arrives, then a truck. The companion model runs both policies:

TEXT

```
FirstFit car -> L-1, truck -> REJECTED
BestFit car -> C-1, truck -> L-1
```

First fit gave the car the only spot a truck could use, and the truck was turned away from a lot with a free spot. Best fit takes the smallest compatible spot, keeping large spots for large vehicles.

Best fit is not universally correct either. If trucks are rare and cars queue at peak, holding large spots empty can reject cars that could have parked. The defensible answer is: best fit as the default, the policy behind an interface, and a note that a real operator would choose based on the arrival mix.

Pricing as a policy with boundaries

Pricing is where off-by-one errors hide. The companion uses a common real-world rule: 15 minutes free, \$3.00 for the first hour, \$2.00 for each started hour after that, capped at \$20.00 per 24 hours.

JAVA

```
record HourlyPricing(long graceMinutes, long firstHourCents, long perHourCents, long dailyCapCents)
    implements PricingPolicy {
    public long priceCents(long minutesParked) {
        if (minutesParked <= graceMinutes) {
            return 0;
        }
        long fullDays = minutesParked / (24 * 60);
        long remainder = minutesParked % (24 * 60);
        long partial = 0;
        if (remainder > 0) {
            long startedHours = (remainder + 59) / 60;
            partial = Math.min(dailyCapCents, firstHourCents + (startedHours - 1) * perHourCents);
        }
        return fullDays * dailyCapCents + partial;
    }
}
```

The boundary table is the test, and every row is checked by the companion:

TEXT

10 min -> \$0.00	inside the grace period
15 min -> \$0.00	grace is inclusive
16 min -> \$3.00	first started hour
60 min -> \$3.00	exactly one hour
61 min -> \$5.00	one minute into the second hour
300 min -> \$11.00	5 hours: 300 + 4 x 200
720 min -> \$20.00	12 hours would be \$25.00; capped
1,440 min -> \$20.00	exactly one day
1,500 min -> \$23.00	one day plus a started hour (\$20 + \$3)

Three decisions are embedded here, and each should be said aloud:

- **Money is long cents.** Never **double**. Master Chapter 5 shows the drift.
- `(remainder + 59) / 60` is ceiling division for positive integers. It avoids `Math.ceil` on a **double**.
- **The cap applies per 24-hour period**, so a second day restarts the first-hour rate. Whether day two should restart at \$3.00 or continue at \$2.00 per hour is a business rule; state your choice and make it one line to change.

Tickets and the close-once invariant

`exit(ticketId)` removes the ticket from the active map and returns the spot to the free list in one synchronized step. Exiting with the same ticket twice throws `IllegalArgumentException`. The companion checks both: a 61-minute stay charges \$5.00, and the second exit is rejected.

Why an exception rather than returning zero? A second exit is a contract violation - a lost or duplicated ticket, a replayed request - and must be visible. Returning zero would silently hide a fraud path.

Concurrent entry, correct by construction

With multiple entrance gates, `park` is called concurrently. The bug every interviewer probes for:

JAVA

```
Optional<Spot> spot = allocation.choose(free, vehicle);    // thread A and B both see C-7
spot.ifPresent(free::remove);                            // both "claim" it
```

Two threads choose the same spot because the choice and the removal are separate steps. The fix is to make them one step. The companion's `ParkingLot.park` is **synchronized**, so choosing and removing happen under one lock, and the ticket is issued inside the same critical section.

The test that proves it runs 8 threads, each making 1,000 park attempts against 500 compact spots:

TEXT

```
8 threads x 1,000 attempts on 500 spots -> 500 parked, 500 distinct spots, 0 free
```

Three independent checks: the number of successful parks equals the number of spots, the set of occupied spot ids has no duplicates, and the free list is empty. Any one of them would catch a double assignment.

Is one lock too slow? Choosing a spot is a scan over a list in memory. Even with thousands of spots, the critical section is microseconds, while a physical gate processes a car every few seconds. Say this, and offer the partitioned design - one lock per floor, with a dispatcher that tries floors in order - only as the answer to "what if this were a 50,000-spot stadium lot with 40 gates?"

Standard extensions

Extension	Where it lands
Multiple floors	<code>Spot.level</code> already exists; a <code>NearestLevelFirst</code> allocation policy
Electric-vehicle charging spots	Compatibility becomes a table; add <code>SpotType.EV_COMPACT</code> ; EV spots accept EVs first, others only when a flag allows
Reservations	A <code>Reservation</code> entity holding a spot for a time window; allocation excludes reserved spots; a no-show releases after a grace period
Weekend or event pricing	A new <code>PricingPolicy</code> , or a <code>CompositePricing</code> choosing by time
Lost ticket	A fixed penalty policy plus a lookup by plate - which means <code>plate</code> must be indexed
Display boards	An observer notified on park and exit; counts by <code>SpotSize</code> are maintained incrementally

WATCH OUT | Failure modes and common mistakes

- **Vehicle class hierarchy with no behaviour.** `Car` extends `Vehicle` adds types without adding meaning. Use an enum until behaviour differs.
- **Compatibility spread across the code.** `if (vehicle == TRUCK && spot.size != LARGE)` in three places. One `fits` method.
- **Pricing with double and `Math.ceil`.** Works for the demo and fails on the boundary tests.
- **Grace period off-by-one.** Is 15 minutes free or charged? Decide, test both sides of the boundary.
- **Check-then-act on spot allocation.** The central concurrency bug of this problem.
- **Ticket closed twice.** Silently returning zero hides duplicate exits.

TRY IT | Interview questions and model answers

Q: Why not a `Vehicle` abstract class? A: Because the only thing that varies by vehicle type is which spots it fits, and that is data, not behaviour. An enum with a minimum spot size says it in one line and keeps the switch exhaustive. If the requirements added behaviour that varies - different entry procedures, different billing - I would introduce a type then.

Q: How do you choose a spot? A: Through an allocation policy. My default is best fit - the smallest compatible spot - because first fit can put a car in the only large spot and reject a truck from a lot with free space. I would note that best fit is not always right: if large spots sit idle while cars queue, the operator might prefer to overflow cars into them.

Q: Two cars arrive at two gates at the same instant. What happens? A: Both call `park`. The method holds the lot's lock across choosing and removing the spot, so the second call sees the first car's spot already gone. I have a test that runs 8 threads against 500 spots and checks exactly 500 succeed with 500 distinct spots.

Q: The lot has 50,000 spots and 40 gates. Is the single lock still acceptable? A: Probably yes, because the critical section is a short in-memory scan and gates are slow. If profiling showed contention, I would partition by floor, give each floor its own lock and free-spot index, and have the entry gate try floors nearest-first. Each invariant is then per floor, which holds because a spot belongs to exactly one floor.

Q: How would you add hourly pricing that varies by time of day? A: A new pricing policy that splits the stay into intervals at rate boundaries and sums each interval at its rate, with the daily cap applied last. The lot does not change; the constructor receives the new policy.

TRY IT | Exercises

- 1 Add EV spots: EVs prefer EV spots, other cars may use them only when no compact spot is free. Write the compatibility table and the allocation policy.
- 2 Write the pricing policy for "first hour free on weekends" without changing `HourlyPricing`.
- 3 Replace the `synchronized` lot with per-floor locks. State the invariant each lock protects and write the concurrent test for it.
- 4 Add reservations. What happens to a reserved spot when the reserving driver is 30 minutes late?

RECAP | Chapter summary

The parking lot is a constrained allocator with billing. Put compatibility on the vehicle type, allocation and pricing behind policies, and money in integer cents. Allocation policy affects which vehicles are rejected, so it is a correctness choice. Make choosing and claiming a spot one atomic step, and prove it with a concurrent test that checks counts and distinctness. Every standard extension should land in a new policy or a small new entity.

TRY IT | Revision checklist

- ☐ I can explain why first fit rejects a truck that best fit admits.
- ☐ I can recite and test the pricing boundaries: grace edge, exact hour, hour plus one minute, cap, day plus one hour.
- ☐ I can write the check-then-act bug and its one-lock fix from memory.
- ☐ I can say where EV spots, reservations, and dynamic pricing plug in.

SDE-2 CORE CHAPTER

Chapter 3 - State Machines: Vending Machine and Elevator

Learning objectives

By the end of this chapter, you should be able to:

- model a problem as an explicit state machine and reject illegal transitions instead of ignoring them;
- place every refusal *before* any irreversible side effect;
- explain why greedy change-making fails with a limited coin inventory and implement the bounded alternative;
- state and test a conservation invariant for money;
- implement LOOK elevator scheduling and state precisely when it is, and is not, better than first-come-first-served; and
- extend a single-car design to a multi-car dispatcher.

WHY IT WORKS | Why this matters at SDE-2

Both problems look like exercises in `enum` syntax. They are actually exercises in *ordering side effects*. A vending machine that dispenses the drink and then discovers it cannot make change has lost either the customer's money or the owner's product. An elevator that serves requests in arrival order is correct and spends more than twice the travel of one that sweeps. The interviewer is watching for whether you find these failure points yourself.

WHY IT WORKS | First-principles model

A **state machine** is a set of states, a set of events, and a transition function. Designing one well means three things:

- 1 **Every state is named.** If behaviour depends on "whether money has been inserted", that is a state, not a boolean field checked in six places.
- 2 **Every transition is explicit, and every other transition is rejected.** Silently ignoring `select` in the idle state hides bugs; returning a clear refusal or throwing makes them visible.
- 3 **Irreversible actions come last.** Validate everything that can fail, then commit. Dispensing, charging a card, and moving a physical car are irreversible.

The vending machine is a *transactional* state machine: it must end every interaction with money conserved. The elevator is a *scheduling* state machine: its state is a position and a direction, and the interesting question is which transition to take next.

Core terminology

- **State:** a named condition that determines which events are legal.
- **Transition:** a legal move from one state to another in response to an event.
- **Guard:** a condition checked before a transition is allowed.
- **Conservation invariant:** a quantity that must balance after every operation, such as inserted money equals money kept plus money returned plus current credit.
- **Bounded change-making:** making change using only coins actually in the machine.
- **FCFS:** first-come-first-served; serve requests in arrival order.
- **LOOK:** keep moving in the current direction while requests remain ahead, then reverse.
- **SCAN:** like LOOK, but travel to the physical end before reversing.

Detailed mechanics

The vending machine as a transactional state machine

TEXT

```

      insert(coin)          select(code): all guards pass
IDLE -----> HAS_MONEY -----> IDLE
                        |         ^         |
                        |         |         |
                        |         |         | insert(coin)
                        |         +---+
                        |         cancel()
                        +-----> IDLE (refund)
                                (dispense + change)

any state --maintenance--> OUT_OF_SERVICE

```

`select` has four guards, and **all of them run before anything leaves the machine**:

JAVA

```

Vend select(String code) {
    if (state != VendingState.HAS_MONEY)           return refuse("insert money first");
    if (product == null || stock.get(code) == 0)     return refuse("sold out");
    if (credit < product.priceCents())               return refuse("insufficient credit");
    Optional<Map<Integer, Integer>> change = makeChange(credit - price, coins);
    if (change.isEmpty())                           return refuse("exact change only");
    // only now: remove coins, decrement stock, record the sale
}

```

The fourth guard is the one most candidates miss. They dispense, then compute change, then discover the machine has no suitable coins. At that point there is no correct outcome.

Why greedy change fails with a real coin box

With unlimited coins of 25, 10, 5, and 1 cents, greedy change - take the largest coin that fits, repeat - is optimal. That fact is well known, and it leads candidates to write greedy change for a vending machine. But a vending machine does not have unlimited coins.

The companion's example: the box holds one 25c and three 10c coins, and 30c change is due.

TEXT

```
change for 30c from {25x1, 10x3}: greedy FAILS, bounded DP {10=3}
```

Greedy takes the quarter, needs 5c, and has none. Three dimes would have worked. The failure is not a contrived corner: over 5,000 random inventories and amounts, greedy failed where change existed in 27 cases.

The fix is bounded change-making: a small dynamic program over the amount, treating each physical coin as a 0/1 item and minimising coin count. For amounts under a few dollars and a few dozen coins it is microseconds. The companion checks, over the same 5,000 trials, that the DP result always sums to the amount, never uses more coins than the box holds, and finds change whenever greedy does.

Seeing the guard order matter

The companion replays a real sequence. The box holds one 25c and three 10c coins; water costs 65c.

TEXT

```
$1.00 for a 65c item with {25x1, 10x3} loaded -> dispensed {25=1, 10=1}  
second $1.00 for the same item -> refused: exact change only  
cancel refunds {100=1}, state IDLE
```

The first sale gives 35c as 25 + 10. The box now holds two dimes and the inserted dollar coin. The second customer needs 35c, which two dimes cannot make, so the machine refuses *before* dispensing, and cancel returns their dollar. This is the "exact change only" light on a real machine, and it exists because of exactly this ordering.

The conservation invariant

After every operation the machine checks:

TEXT

```
inserted = taken for products + returned as change or refund + current credit
```

It is one line, and it catches every bug of the "money appeared or vanished" class: forgetting to reset credit after a sale, refunding twice, or counting a refused sale as taken. Writing it and asserting it after the demo is a strong SDE-2 signal because it shows you think in invariants, not only in happy paths.

Elevator: state, direction, and the next stop

A single elevator car has a floor, a direction (UP, DOWN, IDLE), and a set of pending stops. With a `TreeSet<Integer>` of stops, LOOK is three lines:

JAVA

```
Integer next = direction == Direction.DOWN ? stops.floor(floor) : stops.ceiling(floor);  
if (next == null) {  
    direction = direction == Direction.DOWN ? Direction.UP : Direction.DOWN; // reverse  
}
```

`ceiling` and `floor` are $O(\log n)$ and give "the nearest stop ahead" directly. This is the one place in these problems where choosing a sorted collection is the whole algorithm.

How much better is LOOK, exactly?

Start at floor 5 with requests [1, 9, 2, 8, 3] known up front:

TEXT

```
FCFS travels 30 floors  
LOOK starting UP   visits [8, 9, 3, 2, 1], travels 12 floors  
LOOK starting DOWN visits [3, 2, 1, 8, 9], travels 12 floors
```

FCFS zig-zags across the building: $4 + 8 + 7 + 6 + 5$. LOOK sweeps once each way.

When I first wrote the companion check for this, I asserted that LOOK never travels further than FCFS. The randomized test refuted it: with the car always starting upward, LOOK lost to FCFS in 3 of 2,000 random batches. The smallest case is easy to see:

TEXT

```
start 5, requests [4, 20]: FCFS 17, LOOK up-first 31, LOOK down-first 17
```

Going up first travels to 20 and back down to 4 - $15 + 16$ floors - while FCFS happens to visit the near stop first. The correct claim is narrower: **for a batch of requests known up front, LOOK started in the better direction is optimal**, because any complete tour on a line must reach both extremes, and the cheapest way is to go to the nearer extreme first and then sweep. The companion checks this against the closed-form optimum over the 2,000 batches: best-direction LOOK is optimal in all 2,000 and averages 18.8 floors against FCFS's 52.7.

That correction is worth saying in an interview. Real elevators are *online*: requests arrive while the car moves, and a car already heading up is committed to that direction. LOOK is used because it is fair and bounded - no request waits more than one round trip - not because it is optimal in every instance.

Multiple cars: dispatch is a policy

With several cars, a dispatcher assigns each hall call to one car. It is an allocation policy, exactly like parking-spot allocation:

JAVA

```
interface DispatchPolicy {  
    Car choose(List<Car> cars, int floor, Direction requested);  
}
```

A reasonable default cost function: a car already moving toward the call in the requested direction costs its distance; an idle car costs its distance; a car moving away costs the distance to its last stop plus the distance back. Choose the minimum. Say that production systems also weigh load and use destination dispatch (passengers enter their floor in the lobby), and that both are new policies, not changes to the car.

States the elevator must reject

The car should refuse to move with doors open, refuse to open doors between floors, and enter a maintenance state that ignores new calls but finishes delivering current passengers. These are the transitions interviewers add in the extension round, and an explicit transition table absorbs them as new rows.

WATCH OUT | Failure modes and common mistakes

- **Booleans instead of states.** `hasMoney`, `isDispensing`, `isBroken` checked everywhere; illegal combinations become reachable.
- **Dispense before change.** The canonical vending bug. Guard first, commit last.
- **Greedy change with finite coins.** Correct for unlimited coins, wrong for a coin box.
- **Money in double.** Credit of $0.1 + 0.2$ is not 0.3 .
- **No conservation check.** Money bugs are invisible without one.
- **Overclaiming the elevator algorithm.** LOOK is not always better than FCFS; state the conditions.
- **Treating direction as derived.** Direction is state; a car heading up with no stops above must decide explicitly to reverse or go idle.

TRY IT | Interview questions and model answers

Q: A customer inserts \$1 for a 65c item. When do you check that change is available? A: Before dispensing, as the last guard in `select`. If change cannot be made from the coins actually in the box, the sale is refused with "exact change only" and the customer can cancel for a refund. Once the product drops, there is no correct way to recover.

Q: Isn't greedy change optimal for US coins? A: For unlimited coins, yes. A vending machine has a finite coin box. With one quarter and three dimes, greedy fails to make 30c while three dimes succeed. I use a small

bounded dynamic program instead; it is cheap at these sizes, and in a randomized test greedy failed in 27 of 5,000 inventories where change existed.

Q: How do you know no money is ever lost? A: The machine tracks four totals and checks `inserted == taken + returned + credit` after every operation. It catches missing credit resets, double refunds, and refused sales counted as taken.

Q: Why LOOK over FCFS for the elevator? A: It sweeps instead of zig-zagging, and no request can starve: each is reached within one round trip of the car. On a known batch it is optimal if started in the better direction - it travelled 12 floors against FCFS's 30 in the example. It is not always better with a fixed starting direction: from floor 5 with requests at 4 and 20, going up first travels 31 floors against FCFS's 17. Real cars are online and committed to a direction, so the justification is fairness and bounded waiting, not optimality.

Q: How would you add a second elevator? A: A dispatch policy that assigns each hall call to one car using a cost function - distance for a car coming toward the call in the right direction, distance to its turnaround plus back for a car moving away. The car class does not change; dispatch is a separate policy.

TRY IT | Exercises

- 1 Add a **MAINTENANCE** state to the vending machine that allows coin collection and restocking but rejects sales. Write the transition table.
- 2 Make the vending machine accept card payment. Which guard changes, and where is the new irreversible step?
- 3 Implement SCAN, which travels to the physical end before reversing, and compare its travel with LOOK on the example batch.
- 4 Write the dispatch cost function for three cars and test it with a call that one car is passing.

RECAP | Chapter summary

State machines are about naming states and ordering side effects. Put every refusal before every irreversible action, and check a conservation invariant after each operation. Greedy change-making fails with a finite coin box; bounded change-making does not. For the elevator, LOOK on a sorted set of stops is simple and fair, and it is optimal for a known batch only when started in the better direction - a claim the companion's randomized test forced me to narrow. Multi-car dispatch is an allocation policy.

TRY IT | Revision checklist

- ☐ I can draw the vending state machine and list the four `select` guards in order.
- ☐ I can give the 30c counterexample to greedy change and name the fix.
- ☐ I can write the money conservation invariant.
- ☐ I can implement LOOK with `TreeSet.ceiling` and `floor`.
- ☐ I can state precisely when LOOK is optimal, with the [4, 20] counterexample.

SDE-2 CORE CHAPTER

Chapter 4 - Caches and Rate Limiters: Data Structure Plus Policy

Learning objectives

By the end of this chapter, you should be able to:

- implement an LRU cache two ways - on `LinkedHashMap` and by hand - and explain when each is the right answer in an interview;
- implement an $O(1)$ LFU cache and prove it against a simple reference model;
- explain why a `ReadWriteLock` does not make an access-ordered LRU cache faster;
- add time-to-live expiry with an injected clock, and name the memory leak that lazy expiry alone creates;
- implement fixed-window, sliding-log, and token-bucket rate limiters and show the burst that separates them; and
- choose between them with a stated reason.

WHY IT WORKS | Why this matters at SDE-2

"Design an LRU cache" and "design a rate limiter" are the two most common LLD prompts that are really data-structure prompts. Both have a textbook answer every candidate has memorised. The discrimination comes from the follow-ups: thread safety, eviction ties, expiry, the boundary burst, and time as a dependency. A candidate who writes a correct LRU and then says "and I'd add `synchronized`" without explaining why reads need exclusive access has shown they memorised the structure without understanding it.

WHY IT WORKS | First-principles model

Both problems have the same shape: **a data structure that answers "what now?" in $O(1)$, plus a policy that decides what to forget.**

- An **LRU cache** is a hash map (find by key) plus a recency order (find the victim). Every hit changes the order.
- An **LFU cache** is a hash map plus a frequency order with a recency tie-break.
- A **TTL cache** adds a deadline per entry and a decision about *when* to check it.
- A **rate limiter** is a counter over time. The whole design question is how "over time" is defined: which window, and what happens at its edge.

Time is an input. Code that calls `System.currentTimeMillis()` directly cannot be tested at a boundary; code that takes a `LongSupplier` clock can be tested at the exact millisecond that matters.

Core terminology

- **LRU (least recently used):** evict the entry whose last access is oldest.
- **LFU (least frequently used):** evict the entry with the fewest accesses; break ties by recency.
- **Access order:** a `LinkedHashMap` mode in which `get` moves the entry to the end.
- **TTL (time to live):** a duration after which an entry is treated as absent.
- **Lazy expiry:** removing an expired entry only when it is next read.
- **Fixed window:** count requests per aligned time bucket.
- **Sliding log:** keep a timestamp per request and count those within the last window.
- **Token bucket:** tokens refill at a steady rate up to a capacity; each request spends one.

Detailed mechanics

LRU on LinkedHashMap

The standard library already implements LRU. Two constructor arguments and one override:

JAVA

```
this.map = new LinkedHashMap<>(16, 0.75f, true) {           // true = access order
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
};
```

In an interview, write this first and say so. Then ask whether they want the structure by hand. Many interviewers do - the point of the question is the linked list - but offering the library version first shows you would not hand-roll it in production.

LRU by hand

A `HashMap<K, Node>` for lookup plus a doubly linked list for order, with sentinel `head` and `tail` nodes so that no operation has to special-case an empty list:

JAVA

```
public Optional<V> get(K key) {
    Node<K, V> node = index.get(key);
    if (node == null) {
        return Optional.empty();
    }
    unlink(node);
    linkFront(node);
    return Optional.of(node.value);
}
```

`put` either updates and moves an existing node, or evicts `tail.prev` when full and links a new node at the front. Every operation is $O(1)$.

How do you know it is right? Compare it against the library version. The companion runs 200,000 random `get` and `put` operations over 20 keys against both, with capacity 8, comparing every result and every size:

TEXT

```
hand-rolled LRU vs LinkedHashMap LRU: 0 mismatches over 200,000 ops
```

This is the most useful testing technique in the round: when a simple, obviously correct implementation exists, use it as the oracle for the fast one.

Why reads need an exclusive lock

In an access-ordered `LinkedHashMap`, `get` is a *structural modification* - the Javadoc says so explicitly - because it moves the entry to the end of the list. The same is true of the hand-rolled version: `get` rewires four pointers.

So a `ReadWriteLock` with `get` under the read lock is a data race: two concurrent readers both relink nodes and can corrupt the list. Every operation needs the exclusive lock, which is why the companion's `LinkedHashMapLru` makes `get` *synchronized*. For high read concurrency, the real options are:

- **Sharding:** N independent LRU caches selected by key hash. Eviction becomes approximately LRU per shard.
- **Buffered recency:** record accesses in a lock-free buffer and apply them to the order in batches. This is how Caffeine achieves near-LRU behaviour with concurrent reads.
- **A different policy:** CLOCK, which sets a reference bit on read instead of relinking.

Saying "reads mutate the recency order, so a read lock is wrong; I would shard or buffer" is one of the clearest SDE-2 signals in this problem.

LFU in $O(1)$

LFU needs three maps and one integer:

TEXT

```
values:    key -> value
counts:    key -> access count
buckets:   count -> keys with that count, in recency order (LinkedHashSet)
minFrequency: the smallest count that has a non-empty bucket
```

On access, move the key from bucket C to bucket $C + 1$; if bucket C became empty and C was the minimum, the minimum becomes $C + 1$. On insert when full, evict the first (least recent) key of the minimum bucket, then insert the new key with count 1 and set the minimum to 1. Every step is $O(1)$.

The tie-break matters and is often left unstated. Among keys with the lowest count, the companion evicts the least recently used, because `LinkedHashSet` iterates in insertion order and a key is re-inserted when its count changes.

Checked the same way as LRU, against a reference that simply scans every key for the lowest count and oldest use:

TEXT

0(1) LFU vs linear-scan reference: 0 mismatches over 200,000 ops

LRU and LFU disagree - on purpose

One sequence shows the difference. Capacity 2:

TEXT

put a, put b, get a, get a, get b, put c: LRU keeps a=false b=true | LFU keeps a=true b=false

LRU evicts **a** because **b** was touched more recently. LFU evicts **b** because **a** was used three times and **b** twice. Neither is "correct"; they encode different beliefs about the workload. LRU suits recency-driven access such as sessions; LFU suits stable popularity such as a product catalogue, but it clings to entries that were popular once, which is why production systems often use windowed or aged frequency.

Time to live, and the leak lazy expiry creates

A TTL cache stores a deadline with each entry and treats an entry as absent at or after its deadline. With an injected clock, the boundary is testable to the millisecond:

TEXT

put at t=0 with ttl 1,000 -> get at t=999: hit get at t=1,000: miss

Lazy expiry removes an expired entry when it is read. It is simple and needs no background thread. But entries that are never read again are never removed:

TEXT

1,000 entries written with 10ms TTL, none read: 1,000 still stored after 4s

That is a memory leak with a correct-looking API. The fix is to pair lazy expiry with one of: a size bound (the LRU layer evicts old entries anyway), a periodic sweep over a sample of keys (what Redis does), or a priority queue ordered by deadline. State which one you are using.

Rate limiting: three algorithms, one boundary

Limit: 10 requests per 10 seconds. Fifteen requests arrive at t = 9.9 s and fifteen more at t = 10.0 s - 100 milliseconds apart, straddling a window boundary. The companion runs each limiter:

TEXT

```
fixed window 15 at t=9.9s -> 10, 15 at t=10.0s -> 10, total 20 in 100ms
sliding log   15 at t=9.9s -> 10, 15 at t=10.0s -> 0, total 10 in 100ms
token bucket  15 at t=9.9s -> 10, 15 at t=10.0s -> 0, total 10 in 100ms
```

The **fixed window** resets its counter at $t = 10.0$ s and admits twice the limit within 100 ms. That boundary burst is the reason the question exists.

The **sliding log** keeps each admitted timestamp in a deque, drops those older than the window, and admits only if fewer than 10 remain. It is exact, and costs memory proportional to the limit per client.

The **token bucket** holds up to 10 tokens and refills one per second. It permits a burst up to its capacity, then enforces the average rate:

TEXT

token bucket drained at $t=0$, 10 attempts at $t=3.5$ s -> 3 allowed

The companion stores tokens in thousandths, so refilling one token per second is exactly one milli-token per millisecond and no floating point is involved. Floating-point token counts accumulate rounding error over millions of refills; integer fixed-point does not.

Choosing a limiter

Algorithm	Memory per client	Boundary burst	Use when
Fixed window	one counter	up to 2x limit	approximate limits, very high cardinality
Sliding window counter	two counters	small, approximate	good default at scale
Sliding log	one timestamp per admitted request	none	small limits that must be exact
Token bucket	one counter and one timestamp	up to capacity, by design	APIs that should allow short bursts

The sliding window *counter* - weighting the previous window's count by how much of it overlaps the current sliding window - is the common production compromise: two integers per client and a small error. Mention it as the answer to "the sliding log uses too much memory".

In a distributed deployment, the limiter's state must live somewhere every instance can reach atomically: typically Redis, with the check-and-increment done in one Lua script so that two instances cannot both admit the tenth request. That is the same check-then-act problem as parking-spot allocation, one level up.

WATCH OUT | Failure modes and common mistakes

- **Read lock on an LRU `get`.** Reads mutate recency; it is a race.
- **Forgetting to move on update.** `put` of an existing key must also refresh recency.
- **LFU without a tie-break.** Unspecified eviction among equal counts is non-deterministic and untestable.
- **`System.currentTimeMillis()` inside the cache.** Makes boundary tests impossible; inject a clock.
- **Lazy expiry only.** Correct reads, unbounded memory.
- **Fixed window presented as exact.** It admits up to twice the limit across a boundary.
- **Floating-point tokens.** Drift over time; use fixed-point integers.

TRY IT | Interview questions and model answers

Q: Implement an LRU cache. A: In production I would extend `LinkedHashMap` with access order and override `removeEldestEntry` - three lines. If you want the structure itself: a hash map from key to node, plus a doubly linked list with sentinel head and tail. `get` moves the node to the front; `put` evicts from the back when full. All $O(1)$. I would test it against the `LinkedHashMap` version with random operations.

Q: Make it thread-safe. Can reads share a lock? A: No. In an access-ordered cache, `get` relinks the node, so it is a write. A read lock would let two readers corrupt the list. Every operation takes the exclusive lock. If that is too slow, I would shard by key hash or buffer recency updates, as Caffeine does.

Q: What does LFU evict when two keys have the same count? A: I evict the least recently used among them. Keeping each frequency bucket as a `LinkedHashSet` gives that order for free. I would state the tie-break explicitly because otherwise eviction is non-deterministic.

Q: Why not just use a fixed window counter for rate limiting? A: Because it admits up to twice the limit across a window boundary. With 10 per 10 seconds, 10 requests at 9.9 s and 10 more at 10.0 s all pass - 20 in 100 ms. A sliding log or token bucket admits 10. I would use a token bucket if short bursts are acceptable and a sliding window counter if memory per client matters.

Q: How does the limiter work across 20 API servers? A: The counter moves to shared storage - usually Redis - and the check and increment happen atomically in one script, so two servers cannot both admit the last request. I would add a local token bucket in front as a first line of defence to cut Redis traffic, accepting that the global limit is then slightly soft.

TRY IT | Exercises

- 1 Add `remove(key)` to the hand-rolled LRU and extend the random comparison to cover it.
- 2 Implement the sliding window counter and measure how far it is from the sliding log on the boundary scenario.
- 3 Add a background sweep to the TTL cache that removes at most N expired entries per call, and show the 1,000-entry leak disappearing.
- 4 Make the token bucket per client with a bounded map of clients. What evicts idle clients?

RECAP | Chapter summary

Caches and rate limiters are a fast data structure plus a forgetting policy. Write the library LRU first, then the hand-rolled one, and prove the second against the first. LFU is $O(1)$ with frequency buckets and a stated recency tie-break. Access-ordered reads are writes, so they need exclusive locking or a different concurrency design. Inject time. Lazy expiry needs a bound or a sweep. The fixed window admits a boundary burst of twice its limit; sliding logs and token buckets do not.

TRY IT | Revision checklist

- ☐ I can write both LRU implementations and the random comparison test.
- ☐ I can explain why a read lock is wrong for LRU `get`.
- ☐ I can implement $O(1)$ LFU with `minFrequency` and a recency tie-break.
- ☐ I can name the lazy-expiry leak and two fixes.
- ☐ I can show the fixed-window boundary burst and choose a limiter with a reason.

SDE-2 CORE CHAPTER

Chapter 5 - Split Expenses: Money, Invariants, and Settlement

Learning objectives

By the end of this chapter, you should be able to:

- represent money so that no operation can create or destroy a cent;
- split an amount equally or by percentage and prove that the shares always sum to the total;
- model expenses, balances, and settlements so that the zero-sum invariant holds after every operation;
- explain why minimising the number of settlement transfers is NP-hard, and what to build instead; and
- extend the design to groups, currencies, and edits to past expenses.

WHY IT WORKS | Why this matters at SDE-2

"Design Splitwise" looks like CRUD: users, groups, expenses. The interviewer is actually probing two things. The first is money: whether you know that floating point cannot represent cents and that division creates remainders someone must own. The second is the settlement algorithm: whether you reach for "minimise transactions" and whether you know that its exact form is intractable. Both are domain invariants, and this problem exists to see whether you protect them.

WHY IT WORKS | First-principles model

A shared-expense system is a **ledger with a zero-sum invariant**.

- An **expense** says one person paid an amount and assigns a share of it to each participant.
- A person's **balance** is what they paid minus what they owe. Positive means they are owed money; negative means they owe.
- **Invariant:** across a group, balances always sum to exactly zero. Every expense adds **+paid** to the payer and **-share** to each participant, and the shares sum to the amount paid - *if and only if* splitting never loses or invents a cent.
- A **settlement plan** is a list of transfers that brings every balance to zero.

The first invariant is about arithmetic and the second is about algorithms. Get the first wrong and no algorithm can fix the second.

Core terminology

- **Minor unit:** the smallest unit of a currency - cents for USD, paise for INR, and none for JPY.
- **Remainder:** what is left when an amount does not divide evenly; it must be assigned to someone.
- **Basis point:** one hundredth of a percent; 10,000 basis points make 100%.
- **Largest-remainder method:** assign rounded-down shares, then give the leftover units to the shares with the largest fractional parts.
- **Net balance:** paid minus owed, for one person across all expenses.
- **Zero-sum group:** a subset of people whose balances sum to zero and can settle among themselves.

Detailed mechanics

Floating point cannot hold money

The companion runs the classic demonstrations:

TEXT

```
double: $100 / 3 rounded to cents = 33.33 each, x3 = 99.99
double: 0.1 + 0.2 == 0.3 is false
```

The first is not really a floating-point problem - it is a *remainder* problem that floating point hides. Three shares of \$33.33 lose a cent no matter how the arithmetic is done. The second is the floating-point problem: 0.1 has no exact binary representation, so sums drift.

The fix is to store money as an integer count of minor units - **long** cents - and to make division return shares that provably sum to the total. **BigDecimal** is also correct, but it still needs an explicit rounding mode and still leaves a remainder; it does not remove the need to decide who gets the extra cent.

Minor units are per currency. USD has 2 decimal places, JPY has 0, and some currencies have 3. A **Money** value object should carry its currency and refuse to add amounts in different currencies.

Equal split: the remainder must go somewhere

JAVA

```
static Map<String, Long> splitEqually(long totalCents, List<String> participants) {
    long base = totalCents / participants.size();
    long remainder = totalCents % participants.size();
    Map<String, Long> shares = new LinkedHashMap<>();
    for (int i = 0; i < participants.size(); i++) {
        shares.put(participants.get(i), base + (i < remainder ? 1 : 0));
    }
    return shares;
}
```

TEXT

```
cents: $100 / 3 -> {asha=3334, ben=3333, chen=3333}, sum $100.00
```

The rule "the first **remainder** participants in list order pay one extra cent" is arbitrary, and that is fine - *deterministic* is what matters. State the rule. Some products give the extra cent to the payer, so the payer is never owed a fraction of a cent; others rotate it. Any of these is defensible; an undefined order is not.

Percentage split: the largest-remainder method

Percentages add a second source of rounding. Store them as basis points so that 33.33% is the integer 3,333, and require that they sum to exactly 10,000. Then:

- 1 compute each exact share as `total × bp`, which is in units of 1/10,000 of a cent;
- 2 give each person the rounded-down share;
- 3 hand out the leftover cents one at a time to the largest fractional remainders.

TEXT

```
$100.01 at 33.34/33.33/33.33% -> {asha=3335, ben=3333, chen=3333}
```

Asha's exact share is 3,334.3334 cents; Ben's and Chen's are 3,333.3333. After rounding down, one cent is left over and goes to the largest remainder - Asha's. The companion checks the method over 10,000 random totals and random three-way percentages:

TEXT

```
largest-remainder split: 10,000 random totals and percentages, every sum exact
```

A split function whose output provably sums to its input is the foundation the rest of the design stands on.

Expenses, balances, and the zero-sum invariant

TEXT

```
Expense (entity)    id, paidBy, amount (Money), shares: Map<UserId, Money>, createdAt
Balance            Map<UserId, Money> per group, derived from expenses
SplitStrategy      Map<UserId, Money> split(Money total, List<UserId> people, params)
                   - EqualSplit, PercentageSplit, ExactSplit, SharesSplit
```

SplitStrategy is the policy seam: "split by exact amounts" and "split by shares (2 : 1 : 1)" are new implementations. Every strategy must satisfy the same contract - output sums to input - and one shared test can check every implementation.

The zero-sum check is one line, and the design should assert it after every expense is added:

JAVA

```
check(balances.values().stream().mapToLong(Long::longValue).sum() == 0, "balances sum to zero");
```

Should balances be stored or derived? Derive them from the expense list for correctness, and cache them for speed. The expense list is the source of truth; the balance map is a projection that can always be rebuilt. This is what makes *editing* or *deleting* a past expense safe: reverse its effect on the cache, or rebuild.

Settlement: greedy is fine, optimal is intractable

The standard settlement is greedy: repeatedly have the largest debtor pay the largest creditor as much as possible. It always finishes in at most $n - 1$ transfers for n people with non-zero balances, because each transfer zeroes at least one balance.

It is not always minimal. The companion's example has five balances:

TEXT

```
balances {A=3000, B=2000, C=2000, D=-4000, E=-3000}
greedy settles in 4 transfers: D->A 3000, E->B 2000, D->C 1000, E->C 1000
minimum is 3: {A,E} and {B,C,D} are zero-sum groups
```

The optimum settles $E \rightarrow A$ for \$30 inside one group, and $D \rightarrow B$ and $D \rightarrow C$ for \$20 each inside the other: three transfers. The general rule: if g is the largest number of disjoint zero-sum groups the n non-zero balances can be partitioned into, the minimum settlement is exactly $n - g$ transfers. A group of k people that cannot be split into smaller zero-sum groups needs $k - 1$ transfers, and summing $k - 1$ over the groups gives $n - g$. Maximising g is a partition problem, which is NP-hard.

The companion computes the optimum with an exponential dynamic program over subsets - fine for a dinner with five friends, useless for a group of 60. So the defensible design is:

- use greedy by default - at most $n - 1$ transfers, $O(n \log n)$ with two heaps;
- optionally run the exact search when the group is small (say, at most 12 non-zero balances);
- tell the interviewer *why* you are not promising the minimum.

Candidates who announce "I'll minimise the number of transactions" and then write greedy have claimed something false. Candidates who say "minimum transfers is NP-hard, so I use greedy, which is bounded by $n - 1$ " have shown they know where the difficulty is.

Concurrency and ordering

Two people add expenses to the same group at the same time. If balances are a cached projection, updating them is a read-modify-write, and two concurrent updates can lose one. Options: a per-group lock around "append expense and update balances", or a single-writer queue per group. The per-group lock is the natural answer, because the zero-sum invariant is per group.

Settlements are payments: they must be idempotent. A settlement request carries a client-generated id, and recording the same id twice is a no-op. Otherwise a retried "Ben paid Asha \$33.34" is applied twice and the ledger no longer balances.

Standard extensions

Extension	Where it lands
Split by shares (2 : 1 : 1)	a new <code>SplitStrategy</code> , reusing largest remainder
Multiple currencies	<code>Money</code> carries currency; balances are per currency; conversion is an explicit, dated event
Edit or delete an expense	reverse it against the cached balances, or rebuild from the expense log
Recurring expenses	a scheduler that creates ordinary expenses (chapter 6)
Activity feed	an event published per expense; the feed is a read model
Simplify debts across groups	the same settlement over the union of balances, with the same NP-hard caveat

WATCH OUT | Failure modes and common mistakes

- **double for money.** Drift and lost remainders.
- **Dropping the remainder.** `total / n` for every participant loses up to $n - 1$ cents per expense.
- **Percentages as double.** $33.33\% \times 3$ is 99.99% ; validate integer basis points summing to 10,000.
- **No zero-sum assertion.** The ledger silently stops balancing.
- **Claiming minimal settlement with greedy.** Greedy is bounded, not optimal; minimal is NP-hard.
- **Non-idempotent settlements.** A retried payment is applied twice.

TRY IT | Interview questions and model answers

Q: How do you store amounts? A: As `long` minor units with a currency - cents for USD. Floating point cannot represent most decimal amounts, and even exact arithmetic leaves remainders on division. My split functions return shares that provably sum to the total.

Q: \$100 split three ways. Who pays the extra cent? A: One person pays \$33.34 and two pay \$33.33. The rule I use is the first participant in a fixed order; some products give it to the payer instead. What matters is that the rule is deterministic and stated, and that the shares sum to exactly \$100.00.

Q: How do you split by percentage? A: Percentages as integer basis points summing to 10,000, then the largest-remainder method: round every share down, then give the leftover cents to the largest fractional parts. I test it with random totals and percentages and check that every sum is exact.

Q: Can you minimise the number of payments needed to settle up? A: The exact minimum is NP-hard: it equals the number of non-zero balances minus the maximum number of disjoint zero-sum groups, and finding those groups is a partition problem. I use greedy - largest debtor pays largest creditor - which needs at most $n - 1$ transfers. For small groups I can run an exact search over subsets. With balances +30, +20, +20, -40, -30, greedy uses four transfers and the optimum is three.

Q: Two people add expenses at the same moment. What can go wrong? A: If balances are cached, two read-modify-write updates can lose one. I lock per group around appending the expense and updating the balances, because the invariant is per group. Settlements carry an idempotency key so retries cannot double-apply.

TRY IT | Exercises

- 1 Implement `SharesSplit` for ratios such as 2 : 1 : 1 and reuse the random sum-preservation test.
- 2 Add currency to the ledger and write the rule for a group that spends in two currencies.
- 3 Implement greedy settlement with two priority queues and confirm the $n - 1$ bound on random groups.
- 4 Delete an expense from the middle of a group's history and show that the balances still sum to zero.

RECAP | Chapter summary

Money is integer minor units with a currency, and every split must return shares that sum exactly to the total: equal splits assign the remainder by a stated rule, and percentage splits use basis points and the largest-remainder method. Balances form a zero-sum ledger, derived from expenses and asserted after every change. Greedy settlement is bounded by $n - 1$ transfers but is not minimal; the minimum is NP-hard, and saying so is part of the answer.

TRY IT | Revision checklist

- ☐ I can show why `double` loses money and why remainders exist even without floating point.
- ☐ I can write an equal split and a largest-remainder percentage split that sum exactly.
- ☐ I assert that balances sum to zero after every expense.
- ☐ I can give the five-person counterexample where greedy settlement is not minimal.
- ☐ I can explain why minimum settlement is NP-hard and what I build instead.

SDE-2 CORE CHAPTER

Chapter 6 - Job Scheduler and Notification Service: Lifecycle and Retries

Learning objectives

By the end of this chapter, you should be able to:

- order jobs with dependencies using Kahn's algorithm, with deterministic tie-breaking and a useful cycle error;
- model a job lifecycle as a transition table that rejects illegal moves;
- compute what a cancellation actually cancels;
- design retries with capped exponential backoff and explain why jitter is needed;
- design a notification service with channel strategies, user preferences, templates, fallback, and idempotency; and
- say which parts of both designs change when they move from one process to many.

WHY IT WORKS | Why this matters at SDE-2

The job scheduler and the notification service are the LLD problems closest to real backend work, and interviewers use them to see whether you have operated software, not only written it. Anyone can put jobs in a queue. The SDE-2 questions are: what happens when a job fails, when it fails forever, when it is cancelled while its dependents wait, and when the same message is delivered twice. Those are lifecycle questions, and this chapter treats them as the core of the design.

WHY IT WORKS | First-principles model

Both systems are **work items moving through a lifecycle, with side effects that may fail and may be retried**.

- A **job** has dependencies, a priority, a state, and an attempt count. The scheduler's job is to run each one exactly when its dependencies have succeeded, and to make failure visible.
- A **notification** has a recipient, content, and a set of possible channels. The service's job is to deliver it once, through the best channel that works.

Three principles apply to both:

- 1 **State transitions are explicit.** A job that has succeeded cannot be cancelled; a notification already delivered is not sent again.

- 2 **Retries are bounded and spaced.** Unbounded retries turn a transient outage into a permanent overload.
- 3 **Side effects are idempotent.** At-least-once execution is the realistic guarantee, so repeating an action must be harmless.

Core terminology

- **DAG:** directed acyclic graph; the shape of valid job dependencies.
- **Topological order:** an order in which every job comes after its dependencies.
- **Kahn's algorithm:** repeatedly run a job with no unfinished dependencies.
- **Transitive dependents:** every job that depends, directly or indirectly, on a given job.
- **Exponential backoff:** waiting $\text{base} \times 2^{\text{attempt}}$ between retries, up to a cap.
- **Jitter:** randomising each wait so that many clients do not retry in lockstep.
- **Idempotency key:** an identifier that makes a repeated request a no-op.
- **Fallback channel:** the next delivery channel to try when the preferred one fails.

Detailed mechanics

Ordering jobs: Kahn's algorithm with a priority queue

Kahn's algorithm keeps each job's count of unfinished dependencies. Jobs whose count is zero are *ready*. Running a ready job decrements its dependents' counts, which may make them ready.

The detail that matters in an interview is the data structure holding ready jobs. A plain queue gives *an* order. A **PriorityQueue** ordered by priority and then by id gives a *deterministic* order that honours priority among the jobs that are actually ready - and only among those.

The companion's pipeline: **schema** (priority 5) and **fetch** have no dependencies; **clean** needs **fetch**; **load** needs **clean** and **schema**; **index** and **report** (priority 3) need **load**.

TEXT

```
execution order [schema, fetch, clean, load, report, index]
```

schema runs first because it is ready and has the highest priority. **report** runs before **index** for the same reason. But priority never lets a job jump its dependencies: **report** has priority 3 and still waits for **load**, which has priority 1. Say this explicitly, because "priority scheduling" is often misread as "high priority first, always".

Cycles must be reported, not waited on

If `fetch` is changed to depend on `index`, the graph has a cycle. Kahn's algorithm detects it for free: it finishes with jobs still unprocessed.

TEXT

```
fetch now depends on index -> dependency cycle; stuck jobs [clean, fetch, index, load, report]
```

Note what the stuck set contains. The cycle is `fetch -> clean -> load -> index -> fetch; report` is not on it, but it depends on `load`, so it can never run either. A good error names the stuck jobs; a better one also extracts the cycle itself with a depth-first search so that the user knows which edge to remove. The failure mode to avoid is a scheduler that silently waits forever for jobs that can never become ready.

Validate at submission time, not at run time. Adding an edge that creates a cycle should be rejected when the job is registered.

The lifecycle as a transition table

TEXT

```
PENDING    -> READY, CANCELLED
READY       -> RUNNING, CANCELLED
RUNNING     -> SUCCEEDED, READY (retry), FAILED (retries exhausted)
SUCCEEDED   -> (terminal)
FAILED       -> (terminal)
CANCELLED    -> (terminal)
```

The companion stores this as an `EnumMap<JobState, Set<JobState>>` and rejects any move not in the table. Two consequences:

- A flaky job's history is readable as data:

TEXT

```
flaky job, 3 attempts: [PENDING, READY, RUNNING, READY, RUNNING, READY, RUNNING, SUCCEEDED]
```

- Cancelling a job that already succeeded throws, instead of quietly relabelling completed work.

Whether `RUNNING -> CANCELLED` is allowed is a real design decision. Allowing it requires cooperative cancellation - the running code must check a flag or respond to interruption - and the side effects already performed are not undone. The table above does not allow it; state that choice and what it would take to change.

Cancellation cascades

Cancelling a job means none of its transitive dependents can ever run. They must move to `CANCELLED` too, or they sit in `PENDING` forever:

TEXT

```
cancelling clean also cancels [clean, index, load, report]
```

`schema` and `fetch` are unaffected, because they do not depend on `clean`. The closure is a graph traversal over the reverse edges. The same traversal answers "if this job fails permanently, what else fails?", which is the question an on-call engineer asks first.

Retries: capped exponential backoff, with jitter

Retrying immediately after a failure usually fails again, because whatever broke has not recovered. Exponential backoff spaces retries out:

TEXT

```
backoff base 100ms, cap 5s: [100, 200, 400, 800, 1600, 3200, 5000, 5000]
```

The cap stops the wait from growing without limit. The companion's schedule is deterministic so that it can be tested; production code adds **jitter** - for example, waiting a random duration between zero and the computed delay. Without jitter, a thousand jobs that failed together during an outage all retry at exactly the same instants, and each wave can knock the recovering dependency over again.

Two more rules turn retries from harmful to helpful:

- **Retry only what is retryable.** A timeout or a 503 is transient; a validation error or a 400 will fail identically forever. Classify failures.
- **Bound the attempts and make exhaustion visible.** After the last attempt the job goes to **FAILED**, and in a real system it goes to a dead-letter queue with its error, where someone can see it.

Notification service: channels are strategies

TEXT

Channel (enum)	PUSH, EMAIL, SMS
ChannelSender	Channel channel(); boolean send(userId, body)
Notification (record)	eventId, userId, template, params
Preferences	userId -> ordered List<Channel>
NotificationService	render, choose channels, retry, fall back, deduplicate

Each channel is a `ChannelSender` implementation. Adding WhatsApp is one new class and one enum constant. The service never contains `if (channel == SMS)`.

Preferences, retries, and fallback

A user's preferences are an ordered list: try push, then email. The service tries each channel a bounded number of times before falling back to the next one. The companion's scenario gives push a provider that fails five times and email one that fails once, with two attempts per channel:

TEXT

```
push fails 5x, email fails 1x, 2 attempts per channel -> sent via EMAIL after 4 attempts
```

Two push attempts, then two email attempts, the second of which succeeds. Real systems add per-channel rules: security codes go only by SMS, marketing respects quiet hours, and some notifications must never fall back to a more intrusive channel. Each rule is a policy that filters or reorders the channel list before sending.

Idempotency: the same event twice

Upstream systems deliver events at least once. An "order shipped" event that is redelivered must not produce a second email. The service keys each delivery by (`eventId`, `userId`) and returns the recorded result for a key it has already delivered:

TEXT

```
same event delivered again -> duplicate suppressed, email outbox size 1
```

Two subtleties an interviewer may probe:

- **The key excludes the channel.** If the key included the channel, a redelivered event whose push attempt now succeeds would notify the user twice - once by email earlier, once by push now.
- **Record-then-send versus send-then-record.** If the service records the key and then crashes before sending, the notification is lost. If it sends and crashes before recording, a retry sends it twice. With external providers there is no way to make both atomic; you choose at-least-once (send first, occasional duplicates) or at-most-once (record first, occasional loss) per notification type. Password resets choose at-least-once; marketing chooses at-most-once.

Templates fail early

Rendering fills template variables from parameters. A template with a variable that has no value must fail before anything is sent - "Hi {name}" reaching a customer is a visible defect. The companion's renderer throws on any unfilled template variable, and the check runs before the first channel is tried.

From one process to many

Both designs change in predictable ways when scaled out:

- The ready queue and job states move to durable storage, and "claim a ready job" becomes an atomic update with a lease: set **RUNNING** with an owner and expiry only if the job is still **READY**. An expired lease returns the job to **READY**. This is check-then-act again, now in a database.
- The idempotency map becomes a table with a unique constraint on the key, or a key-value store with set-if-absent and a TTL.
- Delivery moves behind a queue per channel, so a slow SMS provider cannot block email.

WATCH OUT | Failure modes and common mistakes

- **Plain FIFO for ready jobs.** Non-deterministic order across runs; priority ignored.
- **Priority that overrides dependencies.** A high-priority job must still wait for its dependencies.
- **Infinite wait on a cycle.** Detect cycles at submission; name the stuck jobs.
- **Cancelling one job, orphaning its dependents.** Cancel the transitive closure.
- **Immediate, unbounded retries.** Use capped exponential backoff with jitter and a dead-letter destination.
- **Retrying non-retryable failures.** Classify errors first.
- **Idempotency key that includes the channel.** Fallback then causes duplicates.
- `if (channel == ...) in the service.` Channels are strategies.

TRY IT | Interview questions and model answers

Q: How do you order jobs with dependencies and priorities? A: Kahn's algorithm, with ready jobs held in a priority queue ordered by priority and then id. Priority only chooses among jobs whose dependencies have all succeeded; it never lets a job skip a dependency. The id tie-break makes the order deterministic, which makes the scheduler testable.

Q: What happens if someone submits a cycle? A: I reject it at submission. Kahn's algorithm finishes with unprocessed jobs when there is a cycle, and I report the stuck set, which includes the cycle and everything downstream of it. I would also extract the cycle with a DFS so that the error says which edge to remove.

Q: A job fails. What does the scheduler do? A: If the error is retryable, it moves the job from **RUNNING** back to **READY** with the next backoff delay - exponential, capped, with jitter. After the last attempt it moves the job to **FAILED**, sends it to a dead-letter destination, and cancels its transitive dependents, which can no longer run.

Q: How do you avoid sending the same notification twice? A: Every notification carries the event id. I key deliveries by event and user - not by channel, or fallback would cause duplicates - and a repeated key returns the earlier result. Across processes that key lives in a table with a unique constraint. Whether I record before or after sending depends on whether a duplicate or a loss is worse for that notification type.

Q: Push is down. What happens? A: The user's preferences give an ordered channel list. After a bounded number of push attempts the service falls back to the next channel, subject to rules such as "security codes only by SMS". In a real deployment push failures would also trip a circuit breaker, so we stop trying push for everyone while it is down.

TRY IT | Exercises

- 1 Extract and print the actual cycle, not just the stuck set, using DFS with an on-stack marker.
- 2 Add **RUNNING** -> **CANCELLED** with cooperative cancellation. What does a job's code have to do to honour it?
- 3 Add full jitter to the backoff and write a test that remains deterministic by injecting the random source.
- 4 Add quiet hours to the notification service as a policy that defers non-urgent notifications.

RECAP | Chapter summary

Job schedulers and notification services are lifecycle problems. Order jobs with Kahn's algorithm and a priority queue, reject cycles at submission, and name the stuck jobs. Keep the lifecycle in a transition table, cascade cancellation to transitive dependents, and retry with capped, jittered exponential backoff only for retryable errors. Treat channels as strategies, preferences as an ordered fallback list, and deduplicate deliveries by event and user.

TRY IT | Revision checklist

- ☐ I can write Kahn's algorithm with a priority queue and explain what priority can and cannot do.
- ☐ I can explain why **report** is stuck by a cycle it is not part of.
- ☐ I can write the job transition table and defend the **RUNNING** -> **CANCELLED** decision.
- ☐ I can produce a capped backoff schedule and explain why jitter matters.
- ☐ I can explain why the notification idempotency key excludes the channel.

SDE-2 CORE CHAPTER

Chapter 7 - Extension Rounds, Follow-Ups, and Question Bank

Learning objectives

By the end of this chapter, you should be able to:

- predict the extension an interviewer is likely to add to each common problem and design the seam in advance;
- execute an extension under time pressure by touching the fewest types;
- handle the concurrency, persistence, and scale follow-ups that close most rounds;
- apply the method from chapter 1 to two further common prompts, a library system and a board game; and
- score your own mock attempts against the rubric.

WHY IT WORKS | Why this matters at SDE-2

The last 15 to 25 minutes decide most machine-coding rounds. By then almost every candidate has something running. The extension is where a well-factored design and a lucky one separate: one candidate adds a class and changes a constructor argument, the other edits five files and breaks the demo. The follow-up questions that come after - "what if two threads...", "where would this be stored...", "what changes at 100 times the load..." - are where SDE-2 judgement is scored.

WHY IT WORKS | First-principles model

An extension tests one thing: **did you put the seam where the change is?** A seam is a place where behaviour can be replaced without editing the code around it - an interface, a strategy, a table of transitions, an injected clock.

Seams are not free. Every interface is indirection a reader must follow. So the skill is not "add seams everywhere" but "add seams exactly where change is predictable". The good news is that change *is* predictable in this round: interviewers draw extensions from a small, well-known set for each problem.

Core terminology

- **Seam:** a point where behaviour can vary without editing surrounding code.
- **Blast radius:** the number of types a change touches.
- **Open/closed principle:** extend behaviour by adding code, not by modifying working code.
- **Follow-up:** a question after the implementation about concurrency, persistence, scale, or failure.
- **Read model:** a derived view of data optimised for queries, rebuilt from the source of truth.

Detailed mechanics

The extension catalogue

For each problem in this book, the extensions interviewers use most, and the seam that absorbs each one:

Problem	Likely extension	Seam that should already exist
Parking lot	EV spots, reservations, dynamic pricing	<code>AllocationPolicy</code> , <code>PricingPolicy</code> , compatibility table
Vending machine	card payment, maintenance mode	explicit states, payment as a strategy
Elevator	multiple cars, express floors, maintenance	<code>DispatchPolicy</code> , transition table
LRU / LFU cache	TTL, thread safety, eviction listener	injected clock, eviction hook
Rate limiter	per-user tiers, distributed limits	<code>RateLimiter</code> interface, limiter per key
Split expenses	shares split, currencies, edit history	<code>SplitStrategy</code> , <code>Money</code> value, derived balances
Job scheduler	cron schedules, cancellation, priorities	transition table, ready queue comparator
Notification service	new channel, quiet hours, digest	<code>ChannelSender</code> , preference policy

Read this table before every mock. When you model the domain in step 2 of the method, check each likely extension against your types and ask: how many would this touch?

Executing an extension in five minutes

- 1 **Restate the requirement and its edge cases** in one sentence each. "EVs prefer EV spots; other cars may use them only when no compact spot is free."
- 2 **Name the seam** before typing. "This is an allocation policy plus a compatibility change."
- 3 **Write the new type**, not edits to old ones, wherever possible.
- 4 **Wire it** - usually one constructor argument in `main`.
- 5 **Demo it** with one new check that would have failed before.

If the extension genuinely needs a change to an existing type, say so, and say why the seam was not there: "I modelled compatibility as an order of sizes; EV spots are not larger or smaller, they are different, so compatibility becomes a table." That sentence scores better than pretending the change was anticipated.

The follow-up questions that close the round

Almost every round ends with some of these. Prepare one crisp answer for each against every problem you practise.

Concurrency. "Two requests hit this at the same time." Name the invariant, name the object whose lock or atomic operation protects it, and point to the check-then-act you made atomic. For caches, remember that access-ordered reads are writes.

Persistence. "Where does this live when the process restarts?" Entities become rows; the source of truth is usually an append-only log of events (expenses, tickets, job transitions) with derived read models (balances, free-spot counts). The in-memory lock becomes a database constraint, a conditional update, or a lease.

Scale. "One hundred times the load." Identify the partition key that keeps each invariant local - lot floor, expense group, rate-limit key, job id - and shard by it. Invariants that span partitions are the expensive ones; name them.

Failure. "The payment provider or channel is down." Bounded retries with backoff, a fallback, a circuit breaker, and idempotency so that retries are safe.

Testing. "How do you know it works?" Boundary tables for pricing and time, a reference model for data structures, a concurrent test with count and distinctness checks, and invariant assertions after every operation.

Worked prompt: library management system

Scope in writing: search the catalogue; borrow and return copies; place holds; charge late fees. Out of scope: payments, recommendations, inter-library loans.

TEXT

Book (entity)	isbn, title, authors	- the work
Copy (entity)	copyId, isbn, state	- a physical item: AVAILABLE, ON_LOAN, ON_HOLD_SHELF, LOST
Member (entity)	memberId, tier, loans, holds	
Loan (entity)	copy, member, dueDate, returnedAt	
Hold (entity)	isbn, member, placedAt	- queued per isbn, FIFO
LoanPolicy	maxLoans(tier), loanDays(tier)	
FinePolicy	long fineCents(Loan, returnedAt)	

The distinction the interviewer is looking for is **book versus copy**: a hold is placed on a *title*, a loan is of a *copy*. Candidates who merge them cannot answer "three copies, five holds - who gets the returned copy?" The answer: on return, if the title's hold queue is non-empty, the copy goes to **ON_HOLD_SHELF** for the first member in the queue with a pickup deadline; otherwise it becomes **AVAILABLE**.

Invariants: a copy has at most one open loan; a member's open loans never exceed `maxLoans(tier)`; fines use integer cents and a cap. Concurrency: borrowing the last available copy is check-then-act on the copy's state - a conditional update from **AVAILABLE** to **ON_LOAN**.

Worked prompt: board game (snakes and ladders, tic-tac-toe)

These prompts test separation of *rules* from *state* from *players*.

TEXT	
Board (value)	cells, jumps: Map<Integer, Integer> (snakes and ladders are both jumps)
GameState (entity)	positions, currentPlayer, status
Player	id, and a MoveStrategy (human input, bot, scripted for tests)
Dice	an interface - a seeded implementation makes games reproducible
Rules	applyMove(state, roll) -> new state; winner(state)

Two points win the round. First, **snakes and ladders are the same thing** - a jump from one cell to another - so one map models both, and a validator rejects a jump chain that loops. Second, **inject randomness**: a **Dice** interface with a seeded implementation turns a game into a deterministic test. For tic-tac-toe on an N x N board, check the winner incrementally from the last move - its row, column, and two diagonals - in O(N) instead of scanning the board, and say so.

Scoring your own mock

After every timed attempt, score yourself on the five rubric dimensions from chapter 1, 0 to 2 each:

TEXT			
	0	1	2
working critical path	nothing ran	ran, not checked	ran with checks
separation of concerns	one big class	some policies	entities/values/policies distinct
extensibility	extension = rewrite	touched 3+ types	one new type + wiring
concurrency	not considered	mentioned	owner named, check-then-act fixed
communication	silent	answered when asked	drove scope and trade-offs

Eight or more out of ten is a strong SDE-2 attempt. Keep the score sheets; the dimension that stays lowest across attempts is what to practise next.

Question bank

Practise each prompt against the full method, timed at 90 minutes, and score it.

Allocation and pricing

- 1 Parking lot with floors, EV spots, and reservations.
- 2 Movie ticket booking with seat holds that expire after 10 minutes.
- 3 Meeting room scheduler: book, cancel, and find a free room for a time range.
- 4 Hotel booking with room types and overbooking limits.

State machines

- 5 Vending machine with coin inventory and card payment.
- 6 Elevator system with three cars and a dispatcher.
- 7 Traffic light controller with pedestrian requests and an emergency override.

- 8 ATM with cash dispensing by denomination and daily limits.

Data structure plus policy

- 9 LRU cache with TTL and an eviction listener.
- 10 LFU cache with $O(1)$ operations.
- 11 Rate limiter with per-user tiers.
- 12 In-memory key-value store with transactions: **BEGIN, SET, ROLLBACK, COMMIT.**

Domain invariants

- 13 Split expenses with equal, exact, percentage, and shares splits.
- 14 Digital wallet with transfers, idempotency, and statement generation.
- 15 Library management with holds and fines.
- 16 Inventory management with reservations and stock that must never go negative.

Lifecycle and scheduling

- 17 Job scheduler with dependencies, retries, and cancellation.
- 18 Notification service with preferences, fallback, and deduplication.
- 19 Logger library with levels, multiple sinks, and asynchronous writing.
- 20 Pub/sub message broker with topics, consumer groups, and offsets.

Games

- 21 Tic-tac-toe on an $N \times N$ board with an $O(N)$ winner check.
- 22 Snakes and ladders with a seeded dice and loop validation.
- 23 Chess move validation for a subset of pieces.

WATCH OUT | Failure modes and common mistakes

- **Treating extensions as surprises.** They come from a known list; design for it.
- **Editing working code first.** Add a type; wire it; only then refactor if needed.
- **Hand-waving follow-ups.** "We would scale it" is not an answer; name the partition key.
- **Merging book and copy, or rules and state.** Most prompts have one distinction like this that decides the round.
- **Not scoring mocks.** Without a score, practice repeats strengths instead of fixing weaknesses.

TRY IT | Interview questions and model answers

Q: You have ten minutes left and I want EV charging spots. Go. A: EV spots are a new kind of spot, not a bigger one, so compatibility changes from an order to a table: EVs accept EV spots first and compact spots second; other cars accept EV spots only when an overflow flag is set. That is a change to `VehicleType.fits` and a new allocation policy that prefers EV spots for EVs. The lot, ticket, and pricing are untouched. I'll write the policy and add one check that an EV takes the EV spot.

Q: How would you persist the split-expenses ledger? A: Expenses are the source of truth, stored append-only with an id. Balances are a read model derived from them and cached per group. Adding an expense and updating the cached balances happen in one transaction; if they ever disagree, balances are rebuilt from expenses. Settlements are stored with an idempotency key under a unique constraint.

Q: The job scheduler now runs on five machines. What changes? A: Job state moves to a database. A worker claims a job with a conditional update from `READY` to `RUNNING` that sets an owner and a lease expiry; only one update can succeed. A job whose lease expires returns to `READY`. Because a job may then run twice, job side effects must be idempotent.

Q: What is the one mistake you see most often in this round? A: Nothing running at the end. The fix is procedural: a written scope in the first eight minutes, and the critical path executing by minute 45 even if it is crude.

TRY IT | Exercises

- 1 Pick three prompts from the question bank you have never attempted. Do each in 90 minutes and score it.
- 2 For the in-memory key-value store with transactions (prompt 12), design how nested transactions roll back.
- 3 For the movie booking prompt, write the seat-hold state machine and the concurrent test for two users selecting the same seat.

RECAP | Chapter summary

The extension round tests whether your seams are where change happens, and change in this round is predictable: use the extension catalogue. Execute extensions by adding types and wiring them, and say plainly when a seam was missing. Prepare crisp concurrency, persistence, scale, failure, and testing answers for every problem. Practise from the question bank against a timer, and score every attempt against the rubric so that practice targets your weakest dimension.

TRY IT | Revision checklist

- ☐ For each problem in this book, I can name its likely extensions and the seam for each.
- ☐ I can execute an extension in five steps and demo it.
- ☐ I have one crisp answer each for concurrency, persistence, scale, failure, and testing.
- ☐ I can explain book versus copy, and rules versus state.
- ☐ I score every mock attempt out of ten.

SDE-2 CORE CHAPTER

Chapter 8 - Low-Level Design and Machine-Coding Drills

Work these in order. Each drill names the artifact to produce, because "think about it" is not practice. Write your own answer first, then compare with the worked solutions - the gap between the two is the lesson.

Money is always in integer cents. Where a drill asks for code, it should compile and run on Java 21 with no dependencies.

Foundation: method and modeling

D1. Price the boundaries. A lot charges nothing for the first 10 minutes, \$4.00 for the first hour, \$2.50 for each started hour after that, and at most \$25.00 per 24 hours. *Produce:* the charge for 10, 11, 60, 61, 180, 600, 1,440, 1,441, and 2,000 minutes, and the one-line formula for "started hours".

D2. Allocation changes who is rejected. Spots, in declaration order: **L - 1** (large), **C - 1** (compact), **S - 1** (small). Arrivals in order: car, motorcycle, truck, car. *Produce:* the spot each arrival receives under first fit and under best fit, and one sentence on which vehicle best fit could still reject.

D3. Name the concurrency owner. A cinema sells seats. Two users select seat F7 at the same moment. *Produce:* the check-then-act bug in five lines, the fix, the object that owns the lock or atomic operation, and a concurrent test with its three assertions.

State machines

D4. Break greedy change. A vending machine holds one 25c coin and four 10c coins, and owes 40c change. *Produce:* what greedy change does, what bounded change-making returns, and where in **select** the check must run.

D5. Find the money bug. A vending machine records **inserted**, **taken**, and **returned**. After a successful sale it decrements stock, adds the price to **taken**, adds the change to **returned**, and sets the state to **IDLE**. *Produce:* the missing step, the invariant that detects it, and the symptom a customer would see.

D6. Schedule an elevator. The car is at floor 10. Requests known up front: 12, 3, 15, 8. *Produce:* total travel for FCFS, for LOOK starting up, and for LOOK starting down; the visit order for each LOOK run; and the optimal tour length from the closed-form rule.

Data structure plus policy

D7. Trace LRU and LFU. Capacity 3. Operations: put a, put b, put c, get a, put d, get b, get c, get c, put e, get a, get d. *Produce:* for both LRU and LFU (ties broken by least recent use), which key each **put** evicts, whether each **get** hits, and the final contents.

D8. Compare two limiters. Limit: 5 requests per 60 seconds. Requests arrive at $t = 0, 10, 20, 50, 55, 59, 60, 61, 65, 90, 119, 120$ seconds. *Produce:* which requests a fixed window (aligned to multiples of 60) admits, which a sliding log admits, and the totals.

D9. Make a cache thread-safe. *Produce:* the reason a **ReadWriteLock** is wrong for an access-ordered LRU, and two designs that do allow concurrent reads, with the cost of each.

WHY IT WORKS | Domain invariants

D10. Split a bill by percentage. \$99.99 is split 33.33% / 33.33% / 33.34%. *Produce:* the exact share of each person in fractional cents, the largest-remainder result, and one sentence on why the person with the largest percentage does not pay the most.

D11. Settle a group. Balances in dollars: P +40, Q +30, R -20, S -30, T -20. *Produce:* the greedy settlement (largest creditor paid by largest debtor, ties alphabetical), its transfer count, the minimum transfer count, and the zero-sum groups that achieve it.

Lifecycle and scheduling

D12. Order a build pipeline. Jobs (priority in brackets): **build** [2], **lint** [1], **docs** [3], **test** [1] needs build, **package** [1] needs test and lint, **deploy** [1] needs package, **notify** [4] needs deploy and docs. Higher priority runs first among ready jobs; ties break by name. *Produce:* the execution order, the set cancelled when **test** is cancelled, and the backoff schedule for six attempts with base 250 ms and cap 4 s.

D13. Deduplicate notifications. An "order shipped" event is delivered twice. On the first delivery, push failed and email succeeded. On the second delivery, push works. *Produce:* how many notifications the user receives with an idempotency key of (event, user, channel) and with (event, user), and which key you would use.

Challenge

D14. Full mock: seat booking with holds. Design and implement a cinema booking system in 90 minutes: list available seats for a show, hold seats for 10 minutes, confirm a hold into a booking, and release expired holds. *Produce:* the written scope, the domain model, the seat state machine, a runnable critical path, the concurrent test for two users holding the same seat, and your rubric score out of ten.

SDE-2 CORE CHAPTER

Chapter 9 - Low-Level Design and Machine-Coding Drills: Worked Solutions

These are defensible answers, not the only ones. Every number below was computed by a program rather than by hand. Where a solution depends on a business rule the drill does not state, it says which rule it assumed.

Foundation

D1. Price the boundaries

Started hours for a positive number of minutes m is $(m + 59) / 60$ in integer arithmetic.

TEXT

```
10 min -> $0.00    grace is inclusive
11 min -> $4.00    first started hour
60 min -> $4.00    exactly one hour
61 min -> $6.50    4.00 + 1 x 2.50
180 min -> $9.00    4.00 + 2 x 2.50
600 min -> $25.00   10 hours would be $26.50; capped
1,440 min -> $25.00 exactly one day
1,441 min -> $29.00 one day ($25.00) plus a started hour ($4.00)
2,000 min -> $50.00 one day plus 560 minutes: 10 started hours, $26.50, capped at $25.00
```

Rule assumed: the cap applies per 24-hour period and the first-hour rate restarts each day. If the operator wants day two billed at \$2.50 per hour from the start, that is a one-line change to the policy - and a question worth asking.

D2. Allocation changes who is rejected

TEXT

```
first fit:  car -> L-1   motorcycle -> C-1   truck -> REJECTED   car -> REJECTED
best fit:   car -> C-1   motorcycle -> S-1   truck -> L-1       car -> REJECTED
```

First fit wastes both larger spots on smaller vehicles and rejects the truck and the second car. Best fit admits three of four. The last car is rejected under both, because only three spots exist. Best fit can still reject a car when only large spots are held back for trucks that never come - which is why the policy is behind an interface.

D3. Name the concurrency owner

The bug:

JAVA

```
Seat seat = show.seat("F7");
if (seat.state() == SeatState.AVAILABLE) {           // both users see AVAILABLE
    seat.hold(userId, now.plusMinutes(10));           // both succeed
}
```

The fix makes the check and the change one atomic step, owned by the **show**, because the invariant "a seat has at most one active hold or booking" is per show. In memory, a `synchronized hold(seatId, user)` on `Show`, or a `compareAndSet` on the seat's state. In a database, `UPDATE seat SET state = 'HELD', holder = ? WHERE show_id = ? AND seat_id = ? AND state = 'AVAILABLE'`, then check the updated row count is 1.

The test: N threads all try to hold F7 at once. Assert exactly one success, exactly N - 1 failures, and that the seat's holder is the successful user.

State machines

D4. Break greedy change

Greedy takes the 25c coin, still owes 15c, takes one 10c coin, owes 5c, and has no 5c coin: **it fails**. Bounded change-making returns **four 10c coins**. The check runs as the last guard in `select`, before stock is decremented or anything is dispensed; if it fails, the machine refuses with "exact change only".

D5. Find the money bug

The missing step is **resetting credit to zero**. Invariant: `inserted == taken + returned + credit`. After the sale, `credit` still holds the old amount, so the left side is smaller than the right by the price plus change and the check fails at once. The symptom: the next customer finds credit already on the machine and gets a free product.

D6. Schedule an elevator

TEXT

FCFS	10 -> 12 -> 3 -> 15 -> 8	2 + 9 + 12 + 7 = 30 floors
LOOK starting up	visits [12, 15, 8, 3]	5 + 12 = 17 floors
LOOK starting down	visits [8, 3, 12, 15]	7 + 12 = 19 floors
optimal (closed form)	$\min((10-3)+(15-3), (15-10)+(15-3)) = \min(19, 17) = 17$	

The nearer extreme is 15 (five floors away) rather than 3 (seven away), so starting up is optimal. LOOK in the better direction matches the optimum, as the companion shows over 2,000 random batches.

Data structure plus policy

D7. Trace LRU and LFU

TEXT		
op	LRU	LFU (count; ties by least recent)
put a	-	-
put b	-	-
put c	-	-
get a	hit	hit (a=2)
put d	evicts b	evicts b (b=1, older than c=1)
get b	miss	miss
get c	hit	hit (c=2)
get c	hit	hit (c=3)
put e	evicts a	evicts d (d=1 is the only count-1 key)
get a	miss	hit (a=3)
get d	hit	miss
final	{c, d, e}	{a, c, e}

The two policies diverge at **put e**. LRU evicts **a**, last used four operations earlier. LFU evicts **d**, used once, and keeps **a**, used twice already.

D8. Compare two limiters

TEXT													
t (s):	0	10	20	50	55	59	60	61	65	90	119	120	
fixed window	Y	Y	Y	Y	Y	N	Y	Y	Y	Y	Y	Y	total 11
sliding log	Y	Y	Y	Y	Y	N	Y	N	N	Y	Y	Y	total 9

The fixed window resets at $t = 60$, so 60, 61, 65, 90, and 119 all pass in the second window. The sliding log at $t = 61$ still counts 10, 20, 50, 55, and 60 - five within the last 60 seconds - and rejects it. Between $t = 50$ and $t = 65$ the fixed window admits 5 requests in 15 seconds (50, 55, 60, 61, 65); the sliding log admits 3 (50, 55, 60).

D9. Make a cache thread-safe

In an access-ordered LRU, **get** moves the entry to the most-recent position, so it modifies the list. Two readers holding a shared read lock would relink nodes concurrently and corrupt the structure.

Two designs that allow concurrent reads:

- **Sharding.** N independent LRU caches chosen by key hash, each with its own lock. Reads on different shards run in parallel. Cost: eviction is LRU per shard, not global, and hot keys still contend on their shard.
- **Buffered recency.** Reads look up the value in a concurrent map and append the access to a lock-free buffer; a single thread drains the buffer and reorders the list in batches (the Caffeine design). Cost: eviction order lags reads slightly, and the implementation is substantially more complex.

WHY IT WORKS | Domain invariants

D10. Split a bill by percentage

TEXT

```
exact:  9,999 x 3,333 / 10,000 = 3,332.6667 cents    (twice)
        9,999 x 3,334 / 10,000 = 3,333.6666 cents
floors: 3,332 + 3,332 + 3,333 = 9,997    -> 2 cents left over
largest remainders: .6667, .6667, .6666 -> the first two get one cent each
result: 3,333 / 3,333 / 3,333 = 9,999    (each pays $33.33)
```

Everyone pays \$33.33. The largest percentage produced the *smallest* fractional remainder, so the two leftover cents went to the others. That is the method working as designed: it minimises the total rounding error, not the error for the largest share. If the product must guarantee that a larger percentage never pays less, that is a different rule - and it should be written down.

D11. Settle a group

Greedy, in dollars:

TEXT

```
S -> P 30    (P 40 is the largest creditor, S -30 the largest debtor)
R -> Q 20    (Q 30; R and T tie at -20, alphabetical picks R)
T -> P 10
T -> Q 10
4 transfers
```

The minimum is **3**: {Q, S} is a zero-sum group (+30, -30) needing one transfer, and {P, R, T} (+40, -20, -20) needs two - **S -> Q 30, R -> P 20, T -> P 20**. Five non-zero balances in two zero-sum groups need $5 - 2 = 3$ transfers.

Lifecycle and scheduling

D12. Order a build pipeline

TEXT

```
order: docs, build, lint, test, package, deploy, notify
```

Initially ready: **docs** [3], **build** [2], **lint** [1]. **docs** runs first by priority. After **build**, **test** becomes ready with priority 1 - tied with **lint**, so the name decides: **lint**, then **test**. **notify** has the highest priority of all and still runs last, because it waits for **deploy**.

Cancelling **test** cancels {**deploy**, **notify**, **package**, **test**}. **build**, **lint**, and **docs** are unaffected.

TEXT

```
backoff (base 250 ms, cap 4 s), six attempts: 250, 500, 1000, 2000, 4000, 4000    (11.75 s in total)
```

In production, add jitter to each wait.

D13. Deduplicate notifications

- **Key (event, user, channel):** the first delivery records (`shipped`, `u1`, `EMAIL`). The second delivery tries push first; (`shipped`, `u1`, `PUSH`) is a new key and push now works, so the user gets **two** notifications.
- **Key (event, user):** the second delivery finds (`shipped`, `u1`) already delivered and suppresses it; the user gets **one**.

Use (event, user). The channel is *how* the notification was delivered, not *which* notification it was.

Challenge

D14. Full mock: seat booking with holds

Scope. In: list available seats for a show, hold up to N seats for 10 minutes, confirm a hold into a booking, and release expired holds. Out: payments (confirmation assumes payment succeeded), pricing tiers, and multiple cinemas. Say where each would plug in.

Model.

TEXT

Show (entity)	showId, seats: Map<SeatId, Seat> - owns the per-show lock
Seat (entity)	seatId, state, holdId, holdExpiresAt
SeatState (enum)	AVAILABLE, HELD, BOOKED
Hold (entity)	holdId, userId, seatIds, expiresAt
Booking (entity)	bookingId, holdId, userId, seatIds
Clock	injected LongSupplier for expiry tests

Seat state machine.

TEXT

AVAILABLE -> HELD	hold(), all requested seats available (all or nothing)
HELD -> BOOKED	confirm() by the holder, before expiry
HELD -> AVAILABLE	expiry or explicit release
BOOKED ->	(terminal for this design; cancellation would be an extension)

Critical decisions to say aloud.

- Holding several seats is **all or nothing**: check every seat and then mark every seat inside one lock. A partial hold leaves the user with seats they did not want.
- **Expiry is lazy plus swept.** A **HELD** seat past its deadline counts as available on read, and a periodic sweep releases stale holds so that listings are accurate.
- `confirm` checks the hold's owner and expiry inside the same lock, so a hold that expires during confirmation cannot also be re-held by someone else.
- A hold id is returned to the client and makes `confirm` idempotent: confirming the same hold twice returns the same booking.

Concurrent test. Twenty threads call `hold(show, [F7, F8])` at once. Assert exactly one success, nineteen failures, and that F7 and F8 carry the successful hold id. A second test advances the injected clock by 10 minutes and asserts that the seats are holdable again.

Rubric self-score. Score honestly out of ten against chapter 7. Typical first attempts lose points on all-or-nothing holds, which are often implemented seat by seat, and on the expiry race in `confirm`.

SDE-2 CORE CHAPTER

Chapter 10 - Executable Low-Level Design Model

This dependency-free Java 21 model implements all eight problems with injected time and fixed random seeds, and checks every claim in the chapters: allocation outcomes, pricing boundaries, concurrent entry, bounded change-making, money conservation, LOOK scheduling, LRU and LFU against reference models, limiter boundary bursts, exact splits, settlement counts, job ordering, and notification idempotency.

JAVA (continued 1/47)

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.Deque;
import java.util.EnumMap;
import java.util.EnumSet;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.PriorityQueue;
import java.util.Random;
import java.util.Set;
import java.util.TreeMap;
import java.util.TreeSet;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.LongSupplier;

/**
 * Executable model of the eight machine-coding problems in this volume.
 *
 * Every number quoted in the chapters is printed by this program, and every
 * invariant the chapters rely on is checked by it. There are no dependencies:
 * time is injected as a LongSupplier so every scenario is deterministic, and
 * randomized equivalence tests use fixed seeds. Run it with:
 *
 *     javac --release 21 LowLevelDesignModel.java && java -ea LowLevelDesignModel
 *
 * Sections map to chapters:
 * 1. Parking lot: compatibility, allocation policy, pricing, concurrency (ch 2)
 * 2. Vending machine: state machine, change-making, money conservation (ch 3)
 * 3. Elevator: LOOK scheduling against first-come-first-served (ch 3)
```

JAVA (continued 2/47)

```
* 4. LRU and LFU caches, TTL, checked against reference models      (ch 4)
* 5. Rate limiters: fixed window, sliding log, token bucket        (ch 4)
* 6. Split expenses: integer money, remainders, settlement        (ch 5)
* 7. Job scheduler: dependencies, lifecycle, retries, cancellation (ch 6)
* 8. Notification service: channels, preferences, idempotency, fallback (ch 6)
*/
public final class LowLevelDesignModel {

    private LowLevelDesignModel() {}

    static void check(boolean condition, String message) {
        if (!condition) {
            throw new IllegalStateException("check failed: " + message);
        }
    }

    // -----
    // 1. Parking lot
    // -----

    enum VehicleType {
        MOTORCYCLE(SpotSize.SMALL),
        CAR(SpotSize.COMPACT),
        TRUCK(SpotSize.LARGE);

        final SpotSize minimumSpot;

        VehicleType(SpotSize minimumSpot) {
            this.minimumSpot = minimumSpot;
        }

        boolean fits(SpotSize size) {
            return size.ordinal() >= minimumSpot.ordinal();
        }
    }
}
```

JAVA (continued 3/47)

```
enum SpotSize {
    SMALL,
    COMPACT,
    LARGE
}

record Spot(String id, int level, SpotSize size) {}

record Ticket(long ticketId, String plate, Spot spot, long entryMinute) {}

/** The seam that lets allocation rules change without touching the lot. */
interface AllocationPolicy {
    Optional<Spot> choose(List<Spot> freeSpots, VehicleType vehicle);
}

/** Takes the first compatible spot in declaration order. */
static final class FirstFit implements AllocationPolicy {
    @Override
    public Optional<Spot> choose(List<Spot> freeSpots, VehicleType vehicle) {
        return freeSpots.stream().filter(s -> vehicle.fits(s.size())).findFirst();
    }
}

/** Takes the smallest compatible spot, preserving large spots for large vehicles. */
static final class BestFit implements AllocationPolicy {
    @Override
    public Optional<Spot> choose(List<Spot> freeSpots, VehicleType vehicle) {
        return freeSpots.stream()
            .filter(s -> vehicle.fits(s.size()))
            .min(
                Comparator.comparingInt((Spot s) -> s.size().ordinal())
                    .thenComparingInt(Spot::level)
                    .thenComparing(Spot::id));
    }
}
```

JAVA (continued 4/47)

```
interface PricingPolicy {
    long priceCents(long minutesParked);
}

/** Free grace period, first hour, then per started hour, capped per day. */
record HourlyPricing(
    long graceMinutes, long firstHourCents, long perHourCents, long dailyCapCents)
    implements PricingPolicy {
    @Override
    public long priceCents(long minutesParked) {
        if (minutesParked <= graceMinutes) {
            return 0;
        }
        long fullDays = minutesParked / (24 * 60);
        long remainder = minutesParked % (24 * 60);
        long partial = 0;
        if (remainder > 0) {
            long startedHours = (remainder + 59) / 60;
            partial =
                Math.min(dailyCapCents, firstHourCents + (startedHours - 1) * perHourCents);
        }
        return fullDays * dailyCapCents + partial;
    }
}

/** One lock owns allocation: the check and the claim are one atomic step. */
static final class ParkingLot {
    private final List<Spot> free;
    private final Map<Long, Ticket> active = new HashMap<>();
    private final AllocationPolicy allocation;
    private final PricingPolicy pricing;
    private final LongSupplier clockMinutes;
    private long nextTicket = 1;

    ParkingLot(
        List<Spot> spots,
```

JAVA (continued 5/47)

```
        AllocationPolicy allocation,
        PricingPolicy pricing,
        LongSupplier clockMinutes) {
    this.free = new ArrayList<>(spots);
    this.allocation = allocation;
    this.pricing = pricing;
    this.clockMinutes = clockMinutes;
}

synchronized Optional<Ticket> park(String plate, VehicleType vehicle) {
    Optional<Spot> chosen = allocation.choose(free, vehicle);
    if (chosen.isEmpty()) {
        return Optional.empty();
    }
    Spot spot = chosen.get();
    free.remove(spot);
    Ticket ticket = new Ticket(nextTicket++, plate, spot, clockMinutes.getAsLong());
    active.put(ticket.ticketId(), ticket);
    return Optional.of(ticket);
}

synchronized long exit(long ticketId) {
    Ticket ticket = active.remove(ticketId);
    if (ticket == null) {
        throw new IllegalArgumentException("unknown or already-closed ticket " + ticketId);
    }
    free.add(ticket.spot());
    return pricing.priceCents(clockMinutes.getAsLong() - ticket.entryMinute());
}

synchronized int freeCount() {
    return free.size();
}

synchronized Set<String> occupiedSpotIds() {
    Set<String> ids = new HashSet<>();
```

JAVA (continued 6/47)

```

        for (Ticket t : active.values()) {
            ids.add(t.spot().id());
        }
        return ids;
    }

    synchronized int activeCount() {
        return active.size();
    }
}

static List<Spot> spots(int count, SpotSize size) {
    List<Spot> result = new ArrayList<>();
    for (int i = 0; i < count; i++) {
        result.add(new Spot(size.name().charAt(0) + "-" + i, i % 3, size));
    }
    return result;
}

static void parkingLot() throws InterruptedException {
    System.out.println("== 1. Parking lot ==");

    List<Spot> twoSpots =
        List.of(new Spot("L-1", 0, SpotSize.LARGE), new Spot("C-1", 0, SpotSize.COMPACT));
    for (AllocationPolicy policy : List.of(new FirstFit(), new BestFit())) {
        ParkingLot lot =
            new ParkingLot(
                twoSpots, policy, new HourlyPricing(15, 300, 200, 2_000), () -> 0);
        Optional<Ticket> car = lot.park("CAR-1", VehicleType.CAR);
        Optional<Ticket> truck = lot.park("TRUCK-1", VehicleType.TRUCK);
        System.out.printf(
            " %-8s car -> %s, truck -> %s%n",
            policy.getClass().getSimpleName(),
            car.map(t -> t.spot().id()).orElse("REJECTED"),
            truck.map(t -> t.spot().id()).orElse("REJECTED"));
        if (policy instanceof FirstFit) {

```

JAVA (continued 7/47)

```

        check(truck.isEmpty(), "first fit strands the truck");
    } else {
        check(truck.isPresent(), "best fit keeps the large spot for the truck");
    }
}

HourlyPricing pricing = new HourlyPricing(15, 300, 200, 2_000);
long[] durations = {10, 15, 16, 60, 61, 300, 720, 1_440, 1_500};
long[] expected = {0, 0, 300, 300, 500, 1_100, 2_000, 2_000, 2_300};
for (int i = 0; i < durations.length; i++) {
    long price = pricing.priceCents(durations[i]);
    System.out.printf("  %5d min -> %s%n", durations[i], dollars(price));
    check(price == expected[i], "pricing for " + durations[i] + " minutes");
}

long[] now = {0};
ParkingLot clocked =
    new ParkingLot(spots(1, SpotSize.COMPACT), new BestFit(), pricing, () -> now[0]);
Ticket ticket = clocked.park("KA-01", VehicleType.CAR).orElseThrow();
now[0] = 61;
check(clocked.exit(ticket.ticketId()) == 500, "exit charges 61 minutes");
boolean doubleExitRejected = false;
try {
    clocked.exit(ticket.ticketId());
} catch (IllegalArgumentException expectedFailure) {
    doubleExitRejected = true;
}
check(doubleExitRejected, "a ticket can be closed only once");

int spotCount = 500;
int threads = 8;
int attemptsPerThread = 1_000;
ParkingLot contended =
    new ParkingLot(spots(spotCount, SpotSize.COMPACT), new BestFit(), pricing, () -> 0);
AtomicInteger successes = new AtomicInteger();
List<Thread> workers = new ArrayList<>();

```

JAVA (continued 8/47)

```

    for (int t = 0; t < threads; t++) {
        int threadId = t;
        Thread worker =
            new Thread(
                () -> {
                    for (int i = 0; i < attemptsPerThread; i++) {
                        if (contended
                            .park("T" + threadId + "-" + i, VehicleType.CAR)
                            .isPresent()) {
                            successes.incrementAndGet();
                        }
                    }
                }
            );
        workers.add(worker);
        worker.start();
    }
    for (Thread worker : workers) {
        worker.join();
    }
    System.out.printf(
        " %d threads x %d attempts on %d spots -> %d parked, %d distinct spots, %d"
        + " free%n",
        threads,
        attemptsPerThread,
        spotCount,
        successes.get(),
        contended.occupiedSpotIds().size(),
        contended.freeCount());
    check(successes.get() == spotCount, "exactly one success per spot");
    check(contended.occupiedSpotIds().size() == spotCount, "no spot assigned twice");
    check(contended.activeCount() == spotCount && contended.freeCount() == 0, "counts agree");
}

// -----
// 2. Vending machine
// -----

```

JAVA (continued 9/47)

```
enum VendingState {
    IDLE,
    HAS_MONEY,
    OUT_OF_SERVICE
}

record Product(String code, String name, long priceCents) {}

record Vend(
    boolean dispensed, String productCode, Map<Integer, Integer> change, String reason) {}

/**
 * Bounded change-making: minimum coin count using only coins in inventory.
 * Returns empty when no combination exists. Dynamic programming over the
 * amount, expanding each coin into its available copies (0/1 knapsack).
 */
static Optional<Map<Integer, Integer>> makeChange(
    long amountCents, Map<Integer, Integer> inventory) {
    int amount = Math.toIntExact(amountCents);
    int[] best = new int[amount + 1];
    java.util.Arrays.fill(best, Integer.MAX_VALUE);
    best[0] = 0;
    List<Integer> copies = new ArrayList<>();
    for (Map.Entry<Integer, Integer> entry :
        new TreeMap<>(inventory).descendingMap().entrySet()) {
        for (int i = 0; i < entry.getValue(); i++) {
            copies.add(entry.getKey());
        }
    }
    int[][] choice = new int[copies.size()][amount + 1];
    for (int c = 0; c < copies.size(); c++) {
        int coin = copies.get(c);
        choice[c] = new int[amount + 1];
        for (int value = amount; value >= coin; value--) {
            if (best[value - coin] != Integer.MAX_VALUE
```

JAVA (continued 10/47)

```

        && best[value - coin] + 1 < best[value]) {
            best[value] = best[value - coin] + 1;
            choice[c][value] = 1;
        }
    }
}
if (best[amount] == Integer.MAX_VALUE) {
    return Optional.empty();
}
Map<Integer, Integer> result = new TreeMap<>(Comparator.reverseOrder());
int value = amount;
for (int c = copies.size() - 1; c >= 0 && value > 0; c--) {
    if (choice[c][value] == 1 && best[value - copies.get(c)] == best[value] - 1) {
        result.merge(copies.get(c), 1, Integer::sum);
        value -= copies.get(c);
    }
}
check(value == 0, "change reconstruction reaches zero");
return Optional.of(result);
}

/** Greedy change with bounded inventory: the version most candidates write first. */
static Optional<Map<Integer, Integer>> greedyChange(
    long amountCents, Map<Integer, Integer> inventory) {
    long remaining = amountCents;
    Map<Integer, Integer> result = new TreeMap<>(Comparator.reverseOrder());
    for (Map.Entry<Integer, Integer> entry :
        new TreeMap<>(inventory).descendingMap().entrySet()) {
        int coin = entry.getKey();
        int use = (int) Math.min(entry.getValue(), remaining / coin);
        if (use > 0) {
            result.put(coin, use);
            remaining -= (long) use * coin;
        }
    }
    return remaining == 0 ? Optional.of(result) : Optional.empty();
}

```

JAVA (continued 11/47)

```
}

static long value(Map<Integer, Integer> coins) {
    long total = 0;
    for (Map.Entry<Integer, Integer> e : coins.entrySet()) {
        total += (long) e.getKey() * e.getValue();
    }
    return total;
}

static final class VendingMachine {
    private VendingState state = VendingState.IDLE;
    private final Map<String, Product> products = new HashMap<>();
    private final Map<String, Integer> stock = new HashMap<>();
    private final Map<Integer, Integer> coins = new TreeMap<>();
    private final Set<Integer> accepted = Set.of(5, 10, 25, 100);
    private long credit;
    private long takenCents;
    private long insertedCents;
    private long returnedCents;

    void addProduct(Product product, int count) {
        products.put(product.code(), product);
        stock.merge(product.code(), count, Integer::sum);
    }

    void loadCoins(int coin, int count) {
        coins.merge(coin, count, Integer::sum);
    }

    VendingState state() {
        return state;
    }

    void insert(int coin) {
        if (state == VendingState.OUT_OF_SERVICE) {
```

JAVA (continued 12/47)

```

        throw new IllegalStateException("machine is out of service");
    }
    if (!accepted.contains(coin)) {
        throw new IllegalArgumentException("coin not accepted: " + coin);
    }
    coins.merge(coin, 1, Integer::sum);
    credit += coin;
    insertedCents += coin;
    state = VendingState.HAS_MONEY;
}

/** Every refusal happens before anything leaves the machine. */
Vend select(String code) {
    if (state != VendingState.HAS_MONEY) {
        return new Vend(false, code, Map.of(), "insert money first");
    }
    Product product = products.get(code);
    if (product == null || stock.getDefault(code, 0) == 0) {
        return new Vend(false, code, Map.of(), "sold out");
    }
    if (credit < product.priceCents()) {
        return new Vend(false, code, Map.of(), "insufficient credit");
    }
    Optional<Map<Integer, Integer>> change =
        makeChange(credit - product.priceCents(), coins);
    if (change.isEmpty()) {
        return new Vend(false, code, Map.of(), "exact change only");
    }
    change.get().forEach((coin, n) -> coins.merge(coin, -n, Integer::sum));
    stock.merge(code, -1, Integer::sum);
    takenCents += product.priceCents();
    returnedCents += value(change.get());
    credit = 0;
    state = VendingState.IDLE;
    return new Vend(true, code, change.get(), "ok");
}

```

JAVA (continued 13/47)

```
Map<Integer, Integer> cancel() {
    if (state != VendingState.HAS_MONEY) {
        return Map.of();
    }
    Map<Integer, Integer> refund =
        makeChange(credit, coins)
            .orElseThrow(
                () ->
                    new IllegalStateException(
                        "inserted coins must always be refundable"));
    refund.forEach((coin, n) -> coins.merge(coin, -n, Integer::sum));
    returnedCents += value(refund);
    credit = 0;
    state = VendingState.IDLE;
    return refund;
}

boolean moneyConserved() {
    return insertedCents == takenCents + returnedCents + credit;
}

static void vendingMachine() {
    System.out.println();
    System.out.println("== 2. Vending machine ==");

    Map<Integer, Integer> inventory = Map.of(25, 1, 10, 3);
    Optional<Map<Integer, Integer>> greedy = greedyChange(30, inventory);
    Optional<Map<Integer, Integer>> bounded = makeChange(30, inventory);
    System.out.printf(
        "  change for 30c from {25x1, 10x3}: greedy %s, bounded DP %s%n",
        greedy.map(Object::toString).orElse("FAILS"),
        bounded.map(Object::toString).orElse("FAILS"));
    check(greedy.isEmpty(), "greedy takes the quarter and strands 5c");
    check(bounded.isPresent() && bounded.get().equals(Map.of(10, 3)), "three dimes");
}
```

JAVA (continued 14/47)

```
Random random = new Random(42);
int trials = 5_000;
int greedyMisses = 0;
for (int trial = 0; trial < trials; trial++) {
    Map<Integer, Integer> inv = new HashMap<>();
    for (int coin : new int[] {5, 10, 25, 100}) {
        inv.put(coin, random.nextInt(4));
    }
    long amount = 5L * (1 + random.nextInt(40));
    Optional<Map<Integer, Integer>> dp = makeChange(amount, inv);
    Optional<Map<Integer, Integer>> gr = greedyChange(amount, inv);
    if (dp.isPresent()) {
        check(value(dp.get()) == amount, "dp change sums to the amount");
        for (Map.Entry<Integer, Integer> e : dp.get().entrySet()) {
            check(e.getValue() <= inv.getOrDefault(e.getKey(), 0), "dp respects inventory");
        }
    }
    if (gr.isPresent()) {
        check(dp.isPresent(), "dp finds change whenever greedy does");
    }
    if (dp.isPresent() && gr.isEmpty()) {
        greedyMisses++;
    }
}
System.out.printf(
    " random inventories: greedy failed where change existed in %d of %,d trials%n",
    greedyMisses, trials);
check(greedyMisses > 0, "the greedy failure is not a contrived corner");

VendingMachine machine = new VendingMachine();
machine.addProduct(new Product("A1", "Water", 65), 2);
machine.loadCoins(25, 1);
machine.loadCoins(10, 3);
machine.insert(100);
Vend refused = machine.select("A1");
```

JAVA (continued 15/47)

```

        System.out.printf(
            "  $1.00 for a 65c item with {25x1, 10x3} loaded -> %s\n",
            refused.dispensed()
                ? "dispensed " + refused.change()
                : "refused: " + refused.reason());
        check(refused.dispensed(), "35c = 25 + 10 is available");
        check(refused.change().equals(Map.of(25, 1, 10, 1)), "change is 25 + 10");

        machine.insert(100);
        Vend second = machine.select("A1");
        System.out.printf(
            "  second $1.00 for the same item -> %s\n",
            second.dispensed()
                ? "dispensed " + second.change()
                : "refused: " + second.reason());

        check(
            !second.dispensed() && second.reason().equals("exact change only"),
            "only 10+10 left, cannot make 35c; must refuse before dispensing");
        Map<Integer, Integer> refund = machine.cancel();
        System.out.printf("  cancel refunds %s, state %s\n", refund, machine.state());
        check(refund.equals(Map.of(100, 1)), "the inserted dollar comes back");
        check(machine.state() == VendingState.IDLE, "cancel returns to idle");
        check(machine.moneyConserved(), "inserted = taken + returned + credit");
    }

    // -----
    // 3. Elevator scheduling
    // -----

    enum Direction {
        UP,
        DOWN,
        IDLE
    }

    /** LOOK: keep moving while stops remain ahead, then reverse. */

```

JAVA (continued 16/47)

```
static final class LookElevator {
    private final TreeSet<Integer> stops = new TreeSet<>();
    private int floor;
    private Direction direction;

    LookElevator(int startFloor, Direction initial) {
        this.floor = startFloor;
        this.direction = initial;
    }

    void request(int target) {
        if (target != floor) {
            stops.add(target);
        }
    }

    /** Serves every pending stop and returns the visit order. */
    List<Integer> run(int[] travelled) {
        List<Integer> visited = new ArrayList<>();
        while (!stops.isEmpty()) {
            Integer next =
                direction == Direction.DOWN ? stops.floor(floor) : stops.ceiling(floor);
            if (next == null) {
                direction = direction == Direction.DOWN ? Direction.UP : Direction.DOWN;
                continue;
            }
            travelled[0] += Math.abs(next - floor);
            floor = next;
            stops.remove(next);
            visited.add(next);
        }
        direction = Direction.IDLE;
        return visited;
    }
}
```

JAVA (continued 17/47)

```

static int fcfsDistance(int start, int[] requests) {
    int distance = 0;
    int floor = start;
    for (int target : requests) {
        distance += Math.abs(target - floor);
        floor = target;
    }
    return distance;
}

static void elevator() {
    System.out.println();
    System.out.println("== 3. Elevator: LOOK against first-come-first-served ==");
    int start = 5;
    int[] requests = {1, 9, 2, 8, 3};
    int fcfs = fcfsDistance(start, requests);
    System.out.printf(" start %d, requests %s\n", start, java.util.Arrays.toString(requests));
    System.out.printf(" FCFS travels %d floors\n", fcfs);
    check(fcfs == 30, "FCFS distance");
    for (Direction initial : List.of(Direction.UP, Direction.DOWN)) {
        LookElevator car = new LookElevator(start, initial);
        for (int r : requests) {
            car.request(r);
        }
        int[] travelled = {0};
        List<Integer> order = car.run(travelled);
        System.out.printf(
            " LOOK starting %-4s visits %s, travels %d floors\n",
            initial, order, travelled[0]);
        check(travelled[0] == 12, "LOOK distance");
    }

    int[] counter = {4, 20};
    int counterFcfs = fcfsDistance(start, counter);
    int counterLookUp = lookDistance(start, Direction.UP, counter);
    int counterLookDown = lookDistance(start, Direction.DOWN, counter);
}

```

JAVA (continued 18/47)

```
System.out.printf(
    "  start %d, requests %s: FCFS %d, LOOK up-first %d, LOOK down-first %d\n",
    start,
    java.util.Arrays.toString(counter),
    counterFcfs,
    counterLookUp,
    counterLookDown);
check(counterLookUp > counterFcfs, "a fixed initial direction can lose to FCFS");

Random random = new Random(7);
int trials = 2_000;
int fixedUpWorse = 0;
int bestDirectionWorse = 0;
int bestDirectionNotOptimal = 0;
long fcfsTotal = 0;
long lookTotal = 0;
for (int trial = 0; trial < trials; trial++) {
    int s = random.nextInt(20);
    int[] batch = new int[8];
    for (int i = 0; i < batch.length; i++) {
        batch[i] = random.nextInt(20);
    }
    int f = fcfsDistance(s, batch);
    int up = lookDistance(s, Direction.UP, batch);
    int best = Math.min(up, lookDistance(s, Direction.DOWN, batch));
    fcfsTotal += f;
    lookTotal += best;
    if (up > f) {
        fixedUpWorse++;
    }
    if (best > f) {
        bestDirectionWorse++;
    }
    if (best != optimalSweep(s, batch)) {
        bestDirectionNotOptimal++;
    }
}
```

JAVA (continued 19/47)

```

    }
    System.out.printf("  %d random batches of 8 on 20 floors:%n", trials);
    System.out.printf("    LOOK always starting UP loses to FCFS in %d%n", fixedUpWorse);
    System.out.printf(
        "      LOOK in the better direction loses in %d, is optimal in %d%n",
        bestDirectionWorse, trials - bestDirectionNotOptimal);
    System.out.printf(
        "    mean travel %.1f floors (best-direction LOOK) vs %.1f (FCFS)%n",
        (double) lookTotal / trials, (double) fcfsTotal / trials);
    check(fixedUpWorse > 0, "the fixed-direction loss is not a one-off");
    check(
        bestDirectionWorse == 0 && bestDirectionNotOptimal == 0,
        "for a batch known up front, the better of the two sweeps is optimal");
}

static int lookDistance(int start, Direction initial, int[] requests) {
    LookElevator car = new LookElevator(start, initial);
    for (int r : requests) {
        car.request(r);
    }
    int[] travelled = {0};
    car.run(travelled);
    return travelled[0];
}

/** Optimal static tour on a line: reach one extreme, then sweep to the other. */
static int optimalSweep(int start, int[] requests) {
    int low = start;
    int high = start;
    for (int r : requests) {
        low = Math.min(low, r);
        high = Math.max(high, r);
    }
    return Math.min((start - low) + (high - low), (high - start) + (high - low));
}

```

JAVA (continued 20/47)

```
// -----  
// 4. Caches  
// -----  
  
interface Cache<K, V> {  
    Optional<V> get(K key);  
  
    void put(K key, V value);  
  
    int size();  
}  
  
/** LRU on LinkedHashMap: access order plus removeEldestEntry. */  
static final class LinkedHashMapLru<K, V> implements Cache<K, V> {  
    private final LinkedHashMap<K, V> map;  
  
    LinkedHashMapLru(int capacity) {  
        this.map =  
            new LinkedHashMap<>(16, 0.75f, true) {  
                private static final long serialVersionUID = 1L;  
  
                @Override  
                protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {  
                    return size() > capacity;  
                }  
            };  
    }  
  
    @Override  
    public synchronized Optional<V> get(K key) {  
        return Optional.ofNullable(map.get(key));  
    }  
  
    @Override  
    public synchronized void put(K key, V value) {  
        map.put(key, value);  
    }  
}
```

JAVA (continued 21/47)

```
    }

    @Override
    public synchronized int size() {
        return map.size();
    }
}

/** LRU by hand: hash map to nodes of a doubly linked list with sentinels. */
static final class HandRolledLru<K, V> implements Cache<K, V> {
    private static final class Node<K, V> {
        K key;
        V value;
        Node<K, V> prev;
        Node<K, V> next;
    }

    private final int capacity;
    private final Map<K, Node<K, V>> index = new HashMap<>();
    private final Node<K, V> head = new Node<>();
    private final Node<K, V> tail = new Node<>();

    HandRolledLru(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("capacity must be positive");
        }
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    private void unlink(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
}
```

JAVA (continued 22/47)

```
private void linkFront(Node<K, V> node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}

@Override
public Optional<V> get(K key) {
    Node<K, V> node = index.get(key);
    if (node == null) {
        return Optional.empty();
    }
    unlink(node);
    linkFront(node);
    return Optional.of(node.value);
}

@Override
public void put(K key, V value) {
    Node<K, V> node = index.get(key);
    if (node != null) {
        node.value = value;
        unlink(node);
        linkFront(node);
        return;
    }
    if (index.size() == capacity) {
        Node<K, V> eldest = tail.prev;
        unlink(eldest);
        index.remove(eldest.key);
    }
    Node<K, V> fresh = new Node<>();
    fresh.key = key;
    fresh.value = value;
    linkFront(fresh);
}
```

JAVA (continued 23/47)

```
        index.put(key, fresh);
    }

    @Override
    public int size() {
        return index.size();
    }
}

/**
 * O(1) LFU: key -> value, key -> frequency, frequency -> keys in recency
 * order, and the current minimum frequency. Ties evict the least recent.
 */
static final class Lfu<K, V> implements Cache<K, V> {
    private final int capacity;
    private final Map<K, V> values = new HashMap<>();
    private final Map<K, Integer> counts = new HashMap<>();
    private final Map<Integer, LinkedHashSet<K>> buckets = new HashMap<>();
    private int minFrequency;

    Lfu(int capacity) {
        this.capacity = capacity;
    }

    private void touch(K key) {
        int count = counts.get(key);
        LinkedHashSet<K> bucket = buckets.get(count);
        bucket.remove(key);
        if (bucket.isEmpty()) {
            buckets.remove(count);
            if (minFrequency == count) {
                minFrequency = count + 1;
            }
        }
        counts.put(key, count + 1);
        buckets.computeIfAbsent(count + 1, c -> new LinkedHashSet<>()).add(key);
    }
}
```

JAVA (continued 24/47)

```
}

@Override
public Optional<V> get(K key) {
    if (!values.containsKey(key)) {
        return Optional.empty();
    }
    touch(key);
    return Optional.of(values.get(key));
}

@Override
public void put(K key, V value) {
    if (capacity == 0) {
        return;
    }
    if (values.containsKey(key)) {
        values.put(key, value);
        touch(key);
        return;
    }
    if (values.size() == capacity) {
        LinkedHashSet<K> bucket = buckets.get(minFrequency);
        K victim = bucket.iterator().next();
        bucket.remove(victim);
        if (bucket.isEmpty()) {
            buckets.remove(minFrequency);
        }
        values.remove(victim);
        counts.remove(victim);
    }
    values.put(key, value);
    counts.put(key, 1);
    buckets.computeIfAbsent(1, c -> new LinkedHashSet<>()).add(key);
    minFrequency = 1;
}
```

JAVA (continued 25/47)

```
@Override
public int size() {
    return values.size();
}

}

/** Reference LFU: linear scans, obviously correct, used only to check the fast one. */
static final class ReferenceLfu<K, V> implements Cache<K, V> {
    private final int capacity;
    private final Map<K, V> values = new HashMap<>();
    private final Map<K, Integer> counts = new HashMap<>();
    private final Map<K, Long> lastUse = new HashMap<>();
    private long tick;

    ReferenceLfu(int capacity) {
        this.capacity = capacity;
    }

    @Override
    public Optional<V> get(K key) {
        if (!values.containsKey(key)) {
            return Optional.empty();
        }
        counts.merge(key, 1, Integer::sum);
        lastUse.put(key, tick++);
        return Optional.of(values.get(key));
    }

    @Override
    public void put(K key, V value) {
        if (values.containsKey(key)) {
            values.put(key, value);
            counts.merge(key, 1, Integer::sum);
            lastUse.put(key, tick++);
        }
        return;
    }
}
```

JAVA (continued 26/47)

```

    }
    if (values.size() == capacity) {
        K victim = null;
        for (K candidate : values.keySet()) {
            if (victim == null
                || counts.get(candidate) < counts.get(victim)
                || (counts.get(candidate).equals(counts.get(victim))
                    && lastUse.get(candidate) < lastUse.get(victim))) {
                victim = candidate;
            }
        }
        values.remove(victim);
        counts.remove(victim);
        lastUse.remove(victim);
    }
    values.put(key, value);
    counts.put(key, 1);
    lastUse.put(key, tick++);
}

@Override
public int size() {
    return values.size();
}
}

/** Lazy expiry: an entry past its deadline is a miss and is removed on read. */
static final class TtlCache<K, V> {
    private record Entry<V>(V value, long expiresAtMillis) {}

    private final Map<K, Entry<V>> map = new HashMap<>();
    private final LongSupplier clockMillis;

    TtlCache(LongSupplier clockMillis) {
        this.clockMillis = clockMillis;
    }
}

```

JAVA (continued 27/47)

```
void put(K key, V value, long ttlMillis) {
    map.put(key, new Entry<>(value, clockMillis.getAsLong() + ttlMillis));
}

Optional<V> get(K key) {
    Entry<V> entry = map.get(key);
    if (entry == null) {
        return Optional.empty();
    }
    if (clockMillis.getAsLong() >= entry.expiresAtMillis()) {
        map.remove(key);
        return Optional.empty();
    }
    return Optional.of(entry.value());
}

int storedEntries() {
    return map.size();
}

static int compare(
    Cache<Integer, Integer> fast,
    Cache<Integer, Integer> reference,
    long seed,
    int operations,
    int keySpace) {
    Random random = new Random(seed);
    int mismatches = 0;
    for (int op = 0; op < operations; op++) {
        int key = random.nextInt(keySpace);
        if (random.nextBoolean()) {
            if (!fast.get(key).equals(reference.get(key))) {
                mismatches++;
            }
        }
    }
}
```

JAVA (continued 28/47)

```

        }
    } else {
        fast.put(key, op);
        reference.put(key, op);
    }
    if (fast.size() != reference.size()) {
        mismatches++;
    }
}
return mismatches;
}

static void caches() {
    System.out.println();
    System.out.println("== 4. Caches ==");
    int lruMismatches =
        compare(new HandRolledLru<>(8), new LinkedHashMapLru<>(8), 11, 200_000, 20);
    System.out.printf(
        "    hand-rolled LRU vs LinkedHashMap LRU: %d mismatches over 200,000 ops%n",
        lruMismatches);
    check(lruMismatches == 0, "hand-rolled LRU agrees with LinkedHashMap");

    int lfuMismatches = compare(new Lfu<>(8), new ReferenceLfu<>(8), 13, 200_000, 20);
    System.out.printf(
        "    O(1) LFU vs linear-scan reference:      %d mismatches over 200,000 ops%n",
        lfuMismatches);
    check(lfuMismatches == 0, "O(1) LFU agrees with the reference");

    HandRolledLru<String, Integer> lru = new HandRolledLru<>(2);
    Lfu<String, Integer> lfu = new Lfu<>(2);
    for (Cache<String, Integer> cache : List.<Cache<String, Integer>>of(lru, lfu)) {
        cache.put("a", 1);
        cache.put("b", 2);
        cache.get("a");
        cache.get("a");
    }
}

```

JAVA (continued 29/47)

```

        cache.get("b");
        cache.put("c", 3);
    }
    System.out.printf(
        "  put a, put b, get a, get a, get b, put c:  LRU keeps a=%s b=%s | LFU keeps a=%s"
        + " b=%s\n",
        lru.get("a").isPresent(),
        lru.get("b").isPresent(),
        lfu.get("a").isPresent(),
        lfu.get("b").isPresent());
    check(lru.get("a").isEmpty(), "LRU evicts a: b was touched more recently");
    check(
        lfu.get("a").isPresent() && lfu.get("b").isEmpty(),
        "LFU evicts b: a was used more often");

    long[] now = {0};
    TtlCache<String, String> ttl = new TtlCache<>(() -> now[0]);
    ttl.put("session", "alice", 1_000);
    now[0] = 999;
    check(ttl.get("session").isPresent(), "alive before the deadline");
    now[0] = 1_000;
    check(ttl.get("session").isEmpty(), "expired at the deadline");
    check(ttl.storedEntries() == 0, "expired entry removed on read");
    for (int i = 0; i < 1_000; i++) {
        ttl.put("k" + i, "v", 10);
    }
    now[0] = 5_000;
    System.out.printf(
        "  1,000 entries written with 10ms TTL, none read: %d still stored after 4s\n",
        ttl.storedEntries());
    check(ttl.storedEntries() == 1_000, "lazy expiry alone never frees unread keys");
}

// -----
// 5. Rate limiters

```

JAVA (continued 30/47)

```
// -----  
  
interface RateLimiter {  
    boolean tryAcquire(long nowMillis);  
}  
  
static final class FixedWindow implements RateLimiter {  
    private final int limit;  
    private final long windowMillis;  
    private long currentWindow = -1;  
    private int count;  
  
    FixedWindow(int limit, long windowMillis) {  
        this.limit = limit;  
        this.windowMillis = windowMillis;  
    }  
  
    @Override  
    public boolean tryAcquire(long nowMillis) {  
        long window = nowMillis / windowMillis;  
        if (window != currentWindow) {  
            currentWindow = window;  
            count = 0;  
        }  
        if (count < limit) {  
            count++;  
            return true;  
        }  
        return false;  
    }  
}  
  
static final class SlidingLog implements RateLimiter {  
    private final int limit;  
    private final long windowMillis;
```

JAVA (continued 31/47)

```
private final Deque<Long> log = new ArrayDeque<>();

SlidingLog(int limit, long windowMillis) {
    this.limit = limit;
    this.windowMillis = windowMillis;
}

@Override
public boolean tryAcquire(long nowMillis) {
    while (!log.isEmpty() && log.peekFirst() <= nowMillis - windowMillis) {
        log.pollFirst();
    }
    if (log.size() < limit) {
        log.addLast(nowMillis);
        return true;
    }
    return false;
}

}

/** Integer token bucket: tokens held in thousandths so refill needs no floating point. */
static final class TokenBucket implements RateLimiter {
    private final long capacityMilli;
    private final long refillMilliPerMs;
    private long tokensMilli;
    private long lastMillis;

    TokenBucket(int capacity, int tokensPerSecond, long startMillis) {
        this.capacityMilli = capacity * 1_000L;
        this.refillMilliPerMs = tokensPerSecond;
        this.tokensMilli = capacityMilli;
        this.lastMillis = startMillis;
    }

    @Override
```

JAVA (continued 32/47)

```
    public boolean tryAcquire(long nowMillis) {
        long elapsed = Math.max(0, nowMillis - lastMillis);
        tokensMilli = Math.min(capacityMilli, tokensMilli + elapsed * refillMilliPerMs);
        lastMillis = Math.max(lastMillis, nowMillis);
        if (tokensMilli >= 1_000) {
            tokensMilli -= 1_000;
            return true;
        }
        return false;
    }
}

static int burst(RateLimiter limiter, long atMillis, int attempts) {
    int allowed = 0;
    for (int i = 0; i < attempts; i++) {
        if (limiter.tryAcquire(atMillis)) {
            allowed++;
        }
    }
    return allowed;
}

static void rateLimiters() {
    System.out.println();
    System.out.println("== 5. Rate limiters: 10 requests per 10 seconds ==");
    List<RateLimiter> limiters =
        List.of(
            new FixedWindow(10, 10_000),
            new SlidingLog(10, 10_000),
            new TokenBucket(10, 1, 0));
    List<String> names = List.of("fixed window", "sliding log", "token bucket");
    int[] expected = {20, 10, 10};
    for (int i = 0; i < limiters.size(); i++) {
        RateLimiter limiter = limiters.get(i);
        int first = burst(limiter, 9_900, 15);
```

JAVA (continued 33/47)

```

        int second = burst(limiter, 10_000, 15);
        System.out.printf(
            " %-13s 15 at t=9.9s -> %2d, 15 at t=10.0s -> %2d, total %2d in 100ms%n",
            names.get(i), first, second, first + second);
        check(first + second == expected[i], names.get(i) + " boundary behaviour");
    }

    TokenBucket steady = new TokenBucket(10, 1, 0);
    burst(steady, 0, 10);
    int later = burst(steady, 3_500, 10);
    System.out.printf(
        " token bucket drained at t=0, 10 attempts at t=3.5s -> %d allowed%n", later);
    check(later == 3, "3.5 seconds refills three whole tokens");
}

// -----
// 6. Split expenses
// -----

/** Equal split in cents; the remainder goes one cent at a time in participant order. */
static Map<String, Long> splitEqually(long totalCents, List<String> participants) {
    long base = totalCents / participants.size();
    long remainder = totalCents % participants.size();
    Map<String, Long> shares = new LinkedHashMap<>();
    for (int i = 0; i < participants.size(); i++) {
        shares.put(participants.get(i), base + (i < remainder ? 1 : 0));
    }
    return shares;
}

/** Percentage split in basis points, largest-remainder method so shares sum exactly. */
static Map<String, Long> splitByBasisPoints(long totalCents, Map<String, Integer> basisPoints) {
    int totalBp = basisPoints.values().stream().mapToInt(Integer::intValue).sum();
    if (totalBp != 10_000) {
        throw new IllegalArgumentException(

```

JAVA (continued 34/47)

```

        "percentages must sum to 100.00%, got " + totalBp + " bp");
    }
    Map<String, Long> shares = new LinkedHashMap<>();
    Map<String, Long> remainders = new LinkedHashMap<>();
    long assigned = 0;
    for (Map.Entry<String, Integer> e : basisPoints.entrySet()) {
        long exact = totalCents * e.getValue();
        shares.put(e.getKey(), exact / 10_000);
        remainders.put(e.getKey(), exact % 10_000);
        assigned += exact / 10_000;
    }
    List<String> byRemainder = new ArrayList<>(remainders.keySet());
    byRemainder.sort(Comparator.comparingLong((String k) -> remainders.get(k)).reversed());
    for (int i = 0; i < totalCents - assigned; i++) {
        shares.merge(byRemainder.get(i), 1L, Long::sum);
    }
    return shares;
}

record Transfer(String from, String to, long cents) {}

/** Largest creditor is paid by the largest debtor until every balance is zero. */
static List<Transfer> settleGreedy(Map<String, Long> balances) {
    Map<String, Long> b = new TreeMap<>(balances);
    List<Transfer> transfers = new ArrayList<>();
    while (true) {
        String creditor = null;
        String debtor = null;
        for (Map.Entry<String, Long> e : b.entrySet()) {
            if (creditor == null || e.getValue() > b.get(creditor)) {
                creditor = e.getKey();
            }
            if (debtor == null || e.getValue() < b.get(debtor)) {
                debtor = e.getKey();
            }
        }
    }
}

```

JAVA (continued 35/47)

```

    }
    if (creditor == null || b.get(creditor) == 0) {
        return transfers;
    }
    long amount = Math.min(b.get(creditor), -b.get(debtor));
    transfers.add(new Transfer(debtor, creditor, amount));
    b.merge(creditor, -amount, Long::sum);
    b.merge(debtor, amount, Long::sum);
}

/**
 * Minimum number of transfers: n nonzero balances split into the largest
 * number of zero-sum groups g need exactly n - g transfers. Exponential,
 * which is the point: the exact answer is NP-hard, so it is fine for a
 * dinner bill and wrong for a production ledger.
 */
static int minimumTransfers(Map<String, Long> balances) {
    long[] nz =
        balances.values().stream().mapToLong(Long::longValue).filter(v -> v != 0).toArray();
    int n = nz.length;
    long[] sum = new long[1 << n];
    int[] groups = new int[1 << n];
    for (int mask = 1; mask < (1 << n); mask++) {
        int low = Integer.numberOfTrailingZeros(mask);
        sum[mask] = sum[mask & (mask - 1)] + nz[low];
        int best = 0;
        for (int i = 0; i < n; i++) {
            if ((mask >> i & 1) == 1) {
                best = Math.max(best, groups[mask ^ (1 << i)]);
            }
        }
        groups[mask] = best + (sum[mask] == 0 ? 1 : 0);
    }
    return n - groups[(1 << n) - 1];
}

```

JAVA (continued 36/47)

```

}

static void splitExpenses() {
    System.out.println();
    System.out.println("== 6. Split expenses ==");

    double perPerson = Math.round(100.00 / 3 * 100) / 100.0;
    double rebuilt = perPerson * 3;
    System.out.printf(
        "    double: $100 / 3 rounded to cents = %.2f each, x3 = %.2f%n", perPerson, rebuilt);
    check(Math.abs(rebuilt - 100.0) > 0.001, "double rounding loses a cent");
    System.out.printf("    double: 0.1 + 0.2 == 0.3 is %s%n", 0.1 + 0.2 == 0.3);

    Map<String, Long> equal = splitEqually(10_000, List.of("asha", "ben", "chen"));
    System.out.printf(
        "    cents: $100 / 3 -> %s, sum %s%n",
        equal, dollars(equal.values().stream().mapToLong(Long::longValue).sum()));
    check(
        equal.values().stream().mapToLong(Long::longValue).sum() == 10_000,
        "equal split preserves the total");

    Map<String, Integer> thirds = new LinkedHashMap<>();
    thirds.put("asha", 3_334);
    thirds.put("ben", 3_333);
    thirds.put("chen", 3_333);
    Map<String, Long> pct = splitByBasisPoints(10_001, thirds);
    System.out.printf("    $100.01 at 33.34/33.33/33.33%% -> %s%n", pct);
    check(
        pct.values().stream().mapToLong(Long::longValue).sum() == 10_001,
        "percentage split preserves the total");

    Random random = new Random(3);
    for (int trial = 0; trial < 10_000; trial++) {
        long total = 1 + random.nextInt(1_000_000);
        int a = random.nextInt(10_001);
    }
}

```

JAVA (continued 37/47)

```
    int b = random.nextInt(10_001 - a);
    Map<String, Integer> bp = new LinkedHashMap<>();
    bp.put("x", a);
    bp.put("y", b);
    bp.put("z", 10_000 - a - b);
    long sum =
        splitByBasisPoints(total, bp).values().stream()
            .mapToLong(Long::longValue)
            .sum();
    check(sum == total, "largest remainder preserves every total");
}
System.out.println(
    "    largest-remainder split: 10,000 random totals and percentages, every sum exact");

Map<String, Long> balances = new TreeMap<>();
balances.put("A", 3_000L);
balances.put("B", 2_000L);
balances.put("C", 2_000L);
balances.put("D", -4_000L);
balances.put("E", -3_000L);
check(
    balances.values().stream().mapToLong(Long::longValue).sum() == 0,
    "balances sum to zero");
List<Transfer> greedy = settleGreedy(balances);
int optimal = minimumTransfers(balances);
System.out.printf("    balances %s\n", balances);
System.out.printf("    greedy settles in %d transfers: %s\n", greedy.size(), greedy);
System.out.printf("    minimum is %d: {A,E} and {B,C,D} are zero-sum groups\n", optimal);
check(greedy.size() == 4 && optimal == 3, "greedy is not optimal here");

Map<String, Long> after = new TreeMap<>(balances);
for (Transfer t : greedy) {
    after.merge(t.from(), t.cents(), Long::sum);
    after.merge(t.to(), -t.cents(), Long::sum);
}
```

JAVA (continued 38/47)

```
        check(
            after.values().stream().allMatch(v -> v == 0),
            "greedy transfers clear every balance");
    }

    // -----
    // 7. Job scheduler
    // -----

    enum JobState {
        PENDING,
        READY,
        RUNNING,
        SUCCEEDED,
        FAILED,
        CANCELLED;

        private static final Map<JobState, Set<JobState>> ALLOWED = new EnumMap<>(JobState.class);

        static {
            ALLOWED.put(PENDING, EnumSet.of(READY, CANCELLED));
            ALLOWED.put(READY, EnumSet.of(RUNNING, CANCELLED));
            ALLOWED.put(RUNNING, EnumSet.of(SUCCEEDED, READY, FAILED));
            ALLOWED.put(SUCCEEDED, EnumSet.noneOf(JobState.class));
            ALLOWED.put(FAILED, EnumSet.noneOf(JobState.class));
            ALLOWED.put(CANCELLED, EnumSet.noneOf(JobState.class));
        }

        boolean canMoveTo(JobState next) {
            return ALLOWED.get(this).contains(next);
        }
    }

    record Job(String id, int priority, Set<String> dependsOn) {}
```

JAVA (continued 39/47)

```

/** Kahn's algorithm with a priority queue, so ties resolve deterministically. */
static List<String> executionOrder(Map<String, Job> jobs) {
    Map<String, Integer> indegree = new HashMap<>();
    Map<String, List<String>> dependents = new HashMap<>();
    for (Job job : jobs.values()) {
        indegree.putIfAbsent(job.id(), 0);
        for (String dependency : job.dependsOn()) {
            if (!jobs.containsKey(dependency)) {
                throw new IllegalArgumentException(
                    job.id() + " depends on unknown job " + dependency);
            }
            indegree.merge(job.id(), 1, Integer::sum);
            dependents.computeIfAbsent(dependency, d -> new ArrayList<>()).add(job.id());
        }
    }
    PriorityQueue<Job> ready =
        new PriorityQueue<>(
            Comparator.comparingInt(Job::priority).reversed().thenComparing(Job::id));
    indegree.forEach(
        (id, degree) -> {
            if (degree == 0) {
                ready.add(jobs.get(id));
            }
        });
    List<String> order = new ArrayList<>();
    while (!ready.isEmpty()) {
        Job job = ready.poll();
        order.add(job.id());
        for (String next : dependents.getOrDefault(job.id(), List.of())) {
            if (indegree.merge(next, -1, Integer::sum) == 0) {
                ready.add(jobs.get(next));
            }
        }
    }
    if (order.size() != jobs.size()) {

```

JAVA (continued 40/47)

```
        Set<String> stuck = new TreeSet<>(jobs.keySet());
        stuck.removeAll(order);
        throw new IllegalStateException("dependency cycle; stuck jobs " + stuck);
    }
    return order;
}

/** Everything that transitively depends on the cancelled job can never run. */
static Set<String> cancellationClosure(Map<String, Job> jobs, String cancelled) {
    Map<String, List<String>> dependents = new HashMap<>();
    for (Job job : jobs.values()) {
        for (String dependency : job.dependsOn()) {
            dependents.computeIfAbsent(dependency, d -> new ArrayList<>()).add(job.id());
        }
    }
    Set<String> closure = new TreeSet<>();
    Deque<String> work = new ArrayDeque<>(List.of(cancelled));
    while (!work.isEmpty()) {
        String id = work.pop();
        if (closure.add(id)) {
            work.addAll(dependents.getOrDefault(id, List.of()));
        }
    }
    return closure;
}

static List<Long> backoffSchedule(long baseMillis, long capMillis, int attempts) {
    List<Long> delays = new ArrayList<>();
    long delay = baseMillis;
    for (int i = 0; i < attempts; i++) {
        delays.add(Math.min(delay, capMillis));
        delay = Math.min(delay * 2, capMillis);
    }
    return delays;
}
```

JAVA (continued 41/47)

```

static final class TrackedJob {
    private JobState state = JobState.PENDING;
    private final List<JobState> history = new ArrayList<>(List.of(JobState.PENDING));

    void move(JobState next) {
        if (!state.canMoveTo(next)) {
            throw new IllegalStateException("illegal transition " + state + " -> " + next);
        }
        state = next;
        history.add(next);
    }
}

static void jobScheduler() {
    System.out.println();
    System.out.println("== 7. Job scheduler ==");
    Map<String, Job> jobs = new LinkedHashMap<>();
    jobs.put("fetch", new Job("fetch", 1, Set.of()));
    jobs.put("schema", new Job("schema", 5, Set.of()));
    jobs.put("clean", new Job("clean", 1, Set.of("fetch")));
    jobs.put("load", new Job("load", 1, Set.of("clean", "schema")));
    jobs.put("index", new Job("index", 1, Set.of("load")));
    jobs.put("report", new Job("report", 3, Set.of("load")));
    List<String> order = executionOrder(jobs);
    System.out.printf(" execution order %s\n", order);
    check(
        order.equals(List.of("schema", "fetch", "clean", "load", "report", "index")),
        "priority breaks ties among ready jobs only");

    Map<String, Job> cyclic = new LinkedHashMap<>(jobs);
    cyclic.put("fetch", new Job("fetch", 1, Set.of("index")));
    String cycleMessage = "";
    try {
        executionOrder(cyclic);
    }
}

```

JAVA (continued 42/47)

```

    } catch (IllegalStateException expected) {
        cycleMessage = expected.getMessage();
    }
    System.out.printf("  fetch now depends on index -> %s\n", cycleMessage);
    check(cycleMessage.startsWith("dependency cycle"), "cycle reported, not an infinite wait");

    Set<String> cancelled = cancellationClosure(jobs, "clean");
    System.out.printf("  cancelling clean also cancels %s\n", cancelled);
    check(
        cancelled.equals(new TreeSet<>(List.of("clean", "load", "index", "report"))),
        "closure");

    List<Long> delays = backoffSchedule(100, 5_000, 8);
    System.out.printf("  backoff base 100ms, cap 5s: %s\n", delays);
    check(
        delays.equals(List.of(100L, 200L, 400L, 800L, 1_600L, 3_200L, 5_000L, 5_000L)),
        "backoff");

    TrackedJob flaky = new TrackedJob();
    int maxAttempts = 3;
    int failuresBeforeSuccess = 2;
    flaky.move(JobState.READY);
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {
        flaky.move(JobState.RUNNING);
        if (attempt > failuresBeforeSuccess) {
            flaky.move(JobState.SUCCEEDED);
            break;
        }
        flaky.move(attempt == maxAttempts ? JobState.FAILED : JobState.READY);
    }
    System.out.printf("  flaky job, 3 attempts: %s\n", flaky.history);
    check(flaky.state == JobState.SUCCEEDED, "third attempt succeeds");

    TrackedJob done = new TrackedJob();
    done.move(JobState.READY);

```

JAVA (continued 43/47)

```
        done.move(JobState.RUNNING);
        done.move(JobState.SUCCEEDED);
        boolean rejected = false;
        try {
            done.move(JobState.CANCELLED);
        } catch (IllegalStateException expected) {
            rejected = true;
        }
        check(rejected, "a succeeded job cannot be cancelled");
    }

    // -----
    // 8. Notification service
    // -----

    enum Channel {
        PUSH,
        EMAIL,
        SMS
    }

    record Notification(
        String eventId, String userId, String template, Map<String, String> params) {}

    record DeliveryResult(boolean delivered, Channel channel, int attempts, String detail) {}

    interface ChannelSender {
        Channel channel();

        boolean send(String userId, String body);
    }

    /** Fails its first N calls, so retry and fallback are exercised deterministically. */
    static final class FlakySender implements ChannelSender {
        private final Channel channel;
```

JAVA (continued 44/47)

```
private int failuresRemaining;
private final List<String> delivered = new ArrayList<>();

FlakySender(Channel channel, int failures) {
    this.channel = channel;
    this.failuresRemaining = failures;
}

@Override
public Channel channel() {
    return channel;
}

@Override
public boolean send(String userId, String body) {
    if (failuresRemaining > 0) {
        failuresRemaining--;
        return false;
    }
    delivered.add(userId + ": " + body);
    return true;
}

static String render(String template, Map<String, String> params) {
    String out = template;
    for (Map.Entry<String, String> e : params.entrySet()) {
        out = out.replace("{ " + e.getKey() + " ", e.getValue());
    }
    if (out.contains("{")) {
        throw new IllegalArgumentException("unfilled template variable in: " + out);
    }
    return out;
}
```

JAVA (continued 45/47)

```

static final class NotificationService {
    private final Map<Channel, ChannelSender> senders = new EnumMap<>(Channel.class);
    private final Map<String, List<Channel>> preferences = new HashMap<>();
    private final Map<String, DeliveryResult> sent = new HashMap<>();
    private final int attemptsPerChannel;

    NotificationService(List<ChannelSender> senders, int attemptsPerChannel) {
        for (ChannelSender sender : senders) {
            this.senders.put(sender.channel(), sender);
        }
        this.attemptsPerChannel = attemptsPerChannel;
    }

    void prefer(String userId, List<Channel> channels) {
        preferences.put(userId, List.copyOf(channels));
    }

    /** Idempotent per (event, user): a redelivered event returns the first result. */
    DeliveryResult notify(Notification n) {
        String key = n.eventId() + "|" + n.userId();
        DeliveryResult previous = sent.get(key);
        if (previous != null && previous.delivered()) {
            return new DeliveryResult(true, previous.channel(), 0, "duplicate suppressed");
        }
        String body = render(n.template(), n.params());
        int attempts = 0;
        for (Channel channel : preferences.getOrDefault(n.userId(), List.of(Channel.EMAIL))) {
            ChannelSender sender = senders.get(channel);
            if (sender == null) {
                continue;
            }
            for (int i = 0; i < attemptsPerChannel; i++) {
                attempts++;
                if (sender.send(n.userId(), body)) {
                    DeliveryResult result = new DeliveryResult(true, channel, attempts, "sent");

```

JAVA (continued 46/47)

```

        sent.put(key, result);
        return result;
    }
}
}
DeliveryResult failed =
    new DeliveryResult(false, null, attempts, "all channels failed");
sent.put(key, failed);
return failed;
}
}

static void notificationService() {
    System.out.println();
    System.out.println("== 8. Notification service ==");
    FlakySender push = new FlakySender(Channel.PUSH, 5);
    FlakySender email = new FlakySender(Channel.EMAIL, 1);
    NotificationService service = new NotificationService(List.of(push, email), 2);
    service.prefer("u1", List.of(Channel.PUSH, Channel.EMAIL));
    Notification shipped =
        new Notification(
            "order-42-shipped",
            "u1",
            "Order {order} has shipped",
            Map.of("order", "42"));

    DeliveryResult first = service.notify(shipped);
    System.out.printf(
        "    push fails 5x, email fails 1x, 2 attempts per channel -> %s via %s after %d"
        + " attempts%n",
        first.detail(), first.channel(), first.attempts());
    check(
        first.delivered() && first.channel() == Channel.EMAIL && first.attempts() == 4,
        "fallback to email");
}

```

JAVA (continued 47/47)

```
DeliveryResult again = service.notify(shipped);
System.out.printf(
    "  same event delivered again -> %s, email outbox size %d%n",
    again.detail(), email.delivered.size());
check(
    again.detail().equals("duplicate suppressed") && email.delivered.size() == 1,
    "idempotent");

boolean unfilled = false;
try {
    render("Hi {name}, order {order}", Map.of("order", "42"));
} catch (IllegalArgumentException expected) {
    unfilled = true;
}
check(unfilled, "a missing template parameter fails before sending");
check(Objects.equals(render("Order {order}", Map.of("order", "7")), "Order 7"), "render");
}

static String dollars(long cents) {
    return String.format("$%d.%02d", cents / 100, cents % 100);
}

public static void main(String[] args) throws InterruptedException {
    parkingLot();
    vendingMachine();
    elevator();
    caches();
    rateLimiters();
    splitExpenses();
    jobScheduler();
    notificationService();
    System.out.println();
    System.out.println("All low-level design checks passed.");
}
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: method, modeling toolkit, and the parking-lot critical path
- Interview Core: state machines, caches, and rate limiters with invariants checked
- SDE-2 Follow-up: money, settlement, lifecycles, retries, and idempotency
- Challenge: a timed 90-minute mock scored against the rubric

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: SD 03 - Generative AI System Design](#)

[Series index](#)

[Next book: SD 05 - High-Level Design Case Book](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design
System Design	SD 04	Low-Level Design and Machine Coding
System Design	SD 05	High-Level Design Case Book

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: System Design - Book 04 of 05 - Study Step SD-04. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.